Reductions

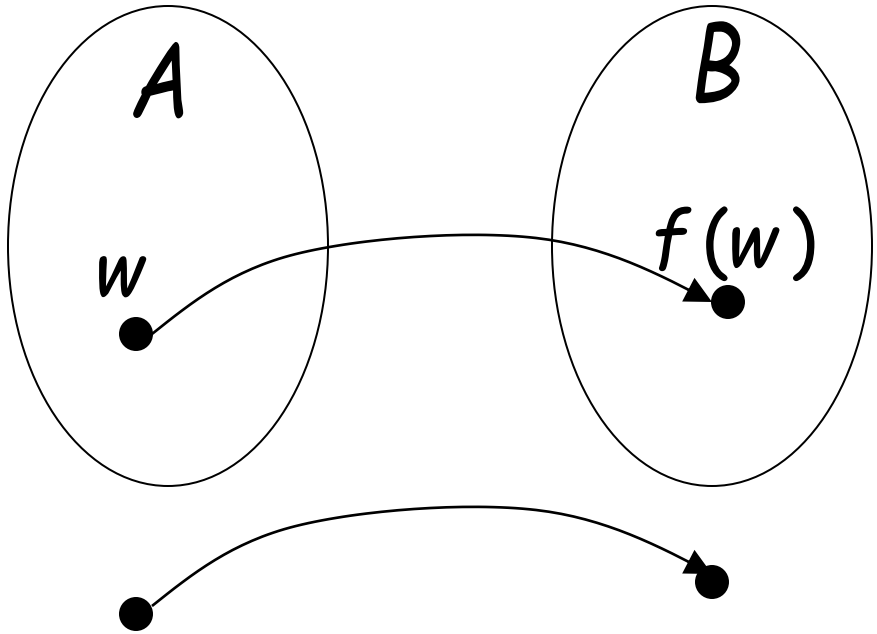
Problem X is reduced to problem Y

If we can solve problem Y

then we can solve problem X

Definition:

Language A
is reduced to
language B



There is a computable
function f (*reduction*) such that:

$$w \in A \iff f(w) \in B$$

Recall:

Computable function f :

There is a deterministic Turing machine M which for any string w computes $f(w)$

Theorem:

If: a: Language A is reduced to B

b: Language B is decidable

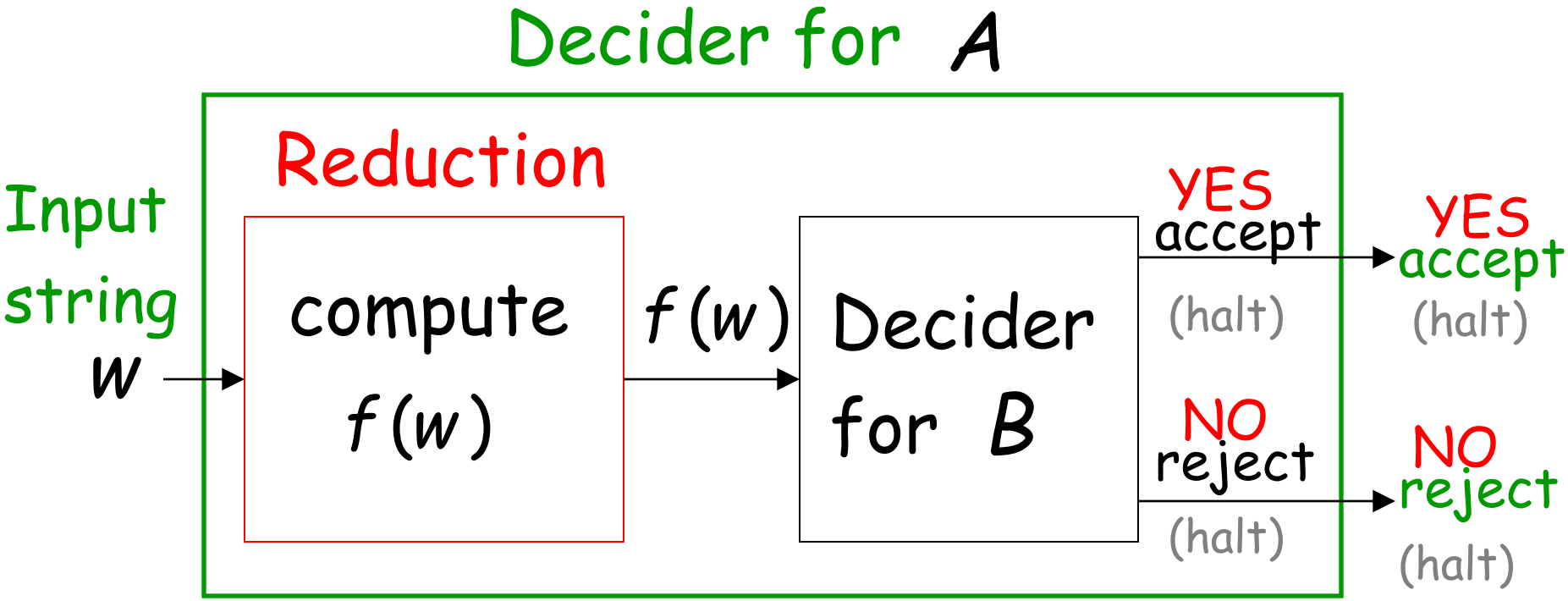
Then: A is decidable

Proof:

Basic idea:

Build the decider for A

using the decider for B



$$w \in A \Leftrightarrow f(w) \in B$$

END OF PROOF

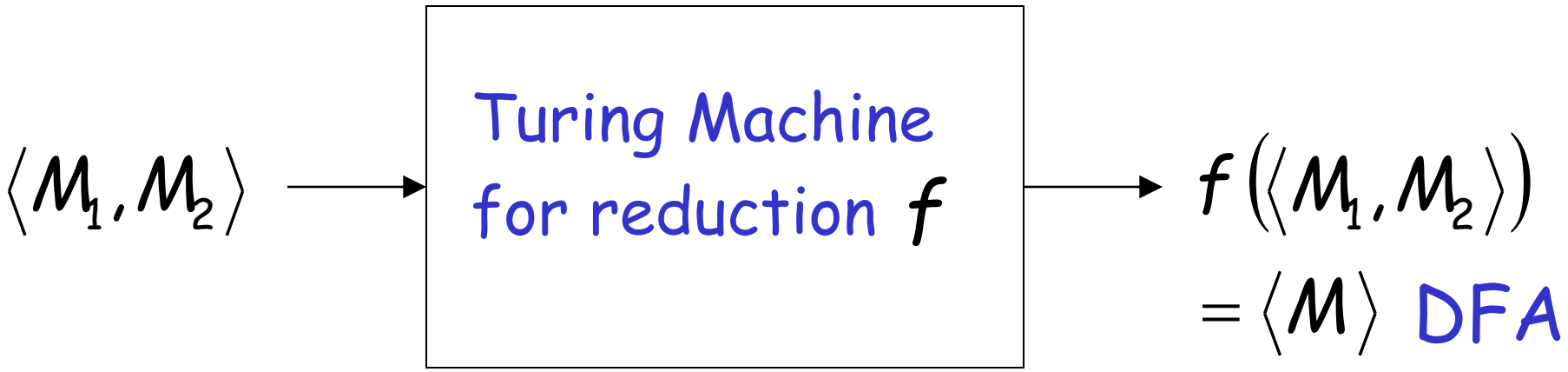
Example:

$$EQUAL_{DFA} = \{\langle M_1, M_2 \rangle : M_1 \text{ and } M_2 \text{ are DFAs that accept the same languages}\}$$

is reduced to:

$$EMPTY_{DFA} = \{\langle M \rangle : M \text{ is a DFA that accepts the empty language } \varnothing\}$$

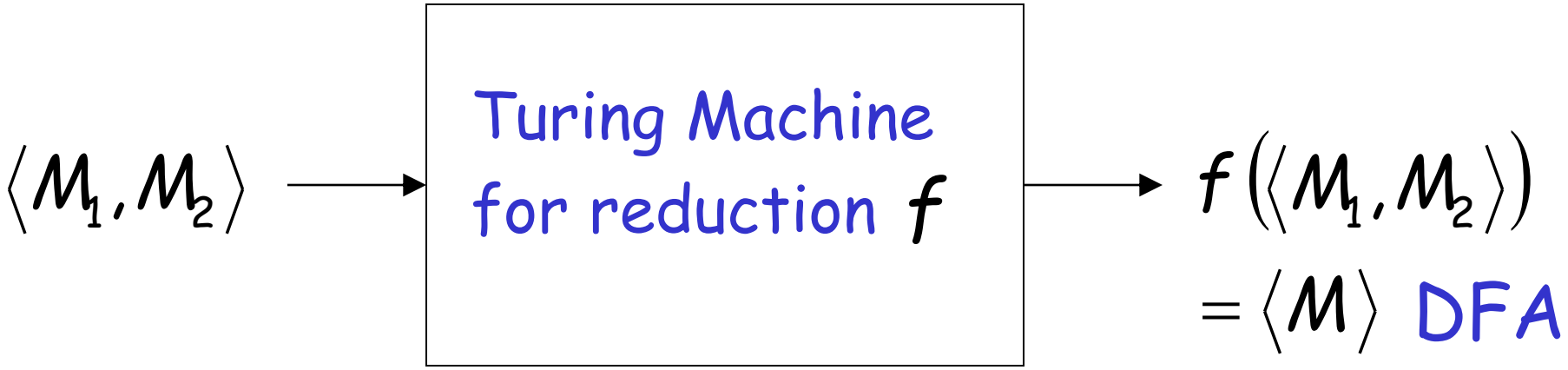
We only need to construct:



$$\langle M_1, M_2 \rangle \in EQUAL_{DFA} \quad \Leftrightarrow \quad \langle M \rangle \in EMPTY_{DFA}$$

Let L_1 be the language of DFA M_1

Let L_2 be the language of DFA M_2

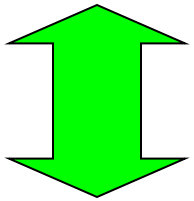


construct DFA M

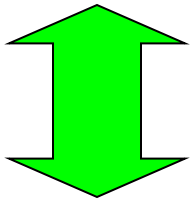
by combining M_1 and M_2 so that:

$$L(M) = (L_1 \cap \overline{L_2}) \cup (\overline{L_1} \cap L_2)$$

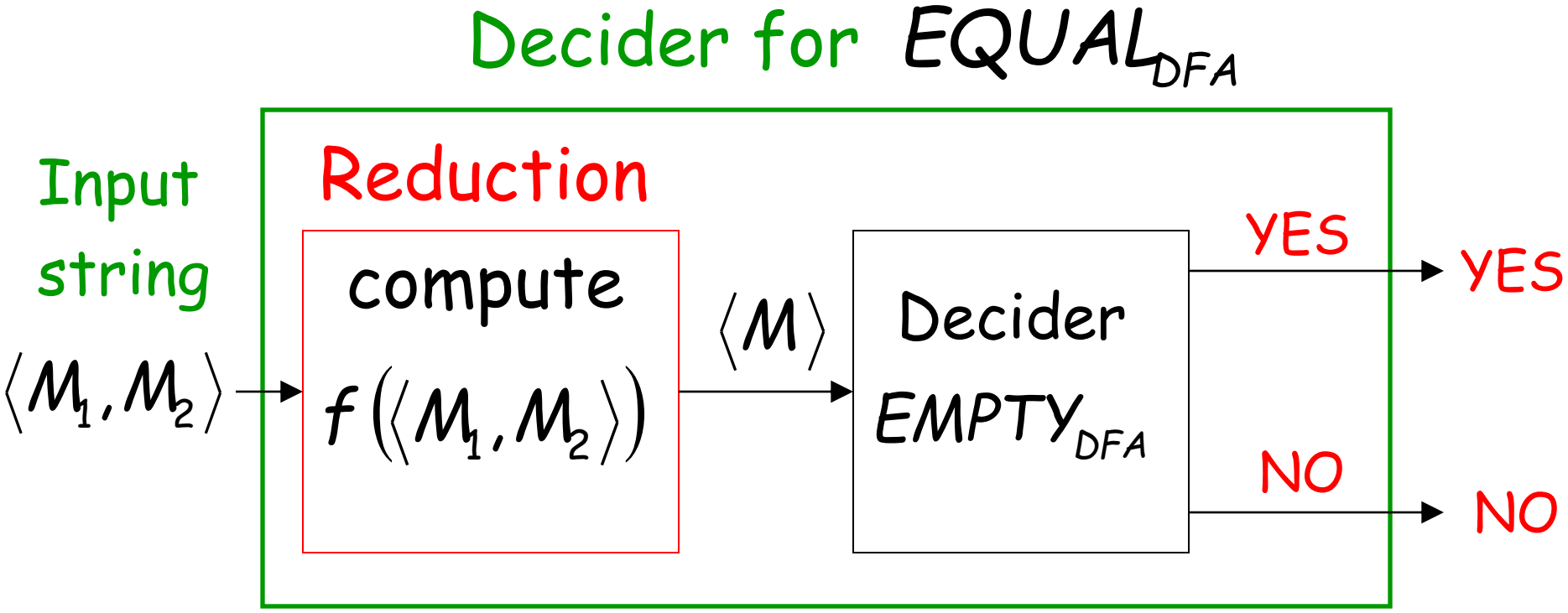
$$L(M) = (L_1 \cap \overline{L_2}) \cup (\overline{L_1} \cap L_2)$$



$$L_1 = L_2 \quad \Leftrightarrow \quad L(M) = \emptyset$$



$$\langle M_1, M_2 \rangle \in EQUAL_{DFA} \quad \Leftrightarrow \quad \langle M \rangle \in EMPTY_{DFA}$$


Decider for $EQUAL_{DFA}$
Input string
$\langle M_1, M_2 \rangle$
Reduction
compute
$f(\langle M_1, M_2 \rangle)$
$\langle M \rangle$
Decider
$EMPTY_{DFA}$
YES
YES
NO
NO

Theorem (version 1):

If: a: Language A is reduced to B

b: Language A is undecidable

Then: B is undecidable

(this is the negation of the previous theorem)

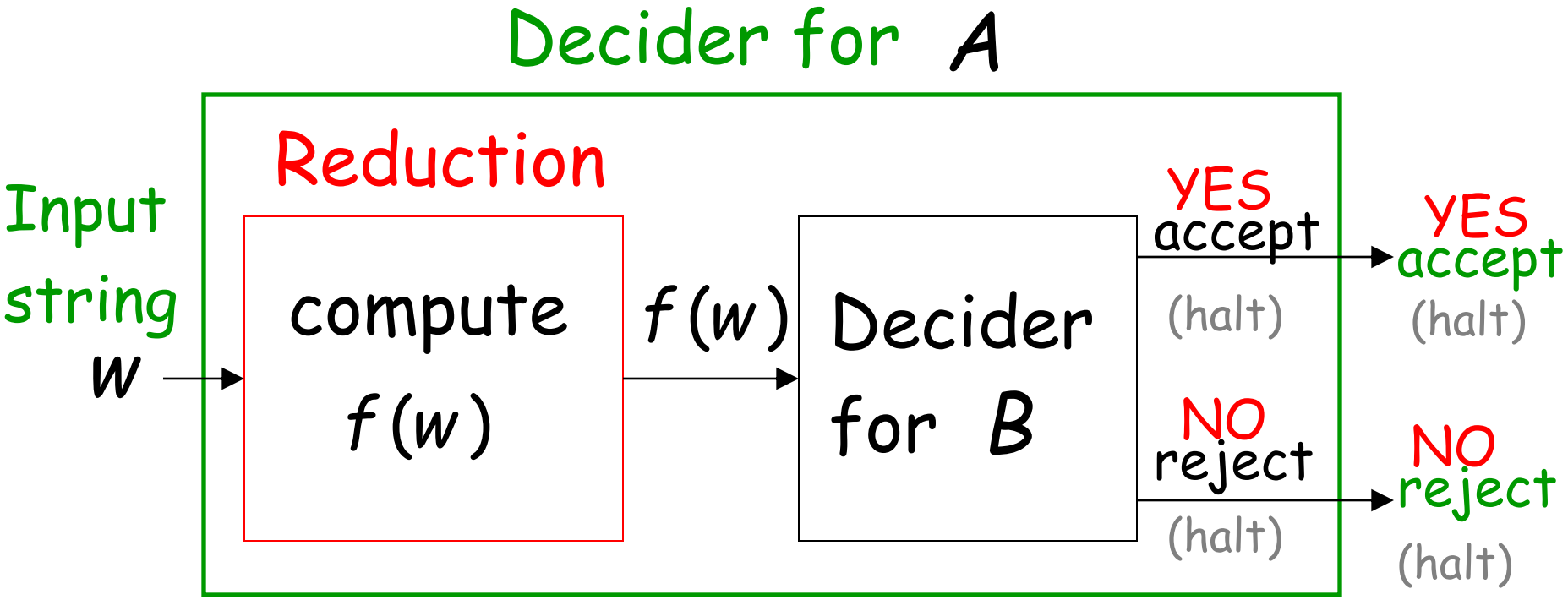
Proof: Suppose B is decidable

Using the decider for B

build the decider for A

Contradiction!

If B is decidable then we can build:



$$w \in A \Leftrightarrow f(w) \in B$$

CONTRADICTION!

END OF PROOF

Observation:

In order to prove
that some language B is undecidable
we only need to reduce a
known undecidable language A
to B

State-entry problem

Input:
- Turing Machine M
- State q
- String w

Question: Does M enter state q while processing input string w ?

Corresponding language:

$$STATE_{TM} = \{\langle M, w, q\rangle : M \text{ is a Turing machine that enters state } q \text{ on input string } w\}$$

Theorem: $STATE_{TM}$ is undecidable

(state-entry problem is unsolvable)

Proof:

Reduce

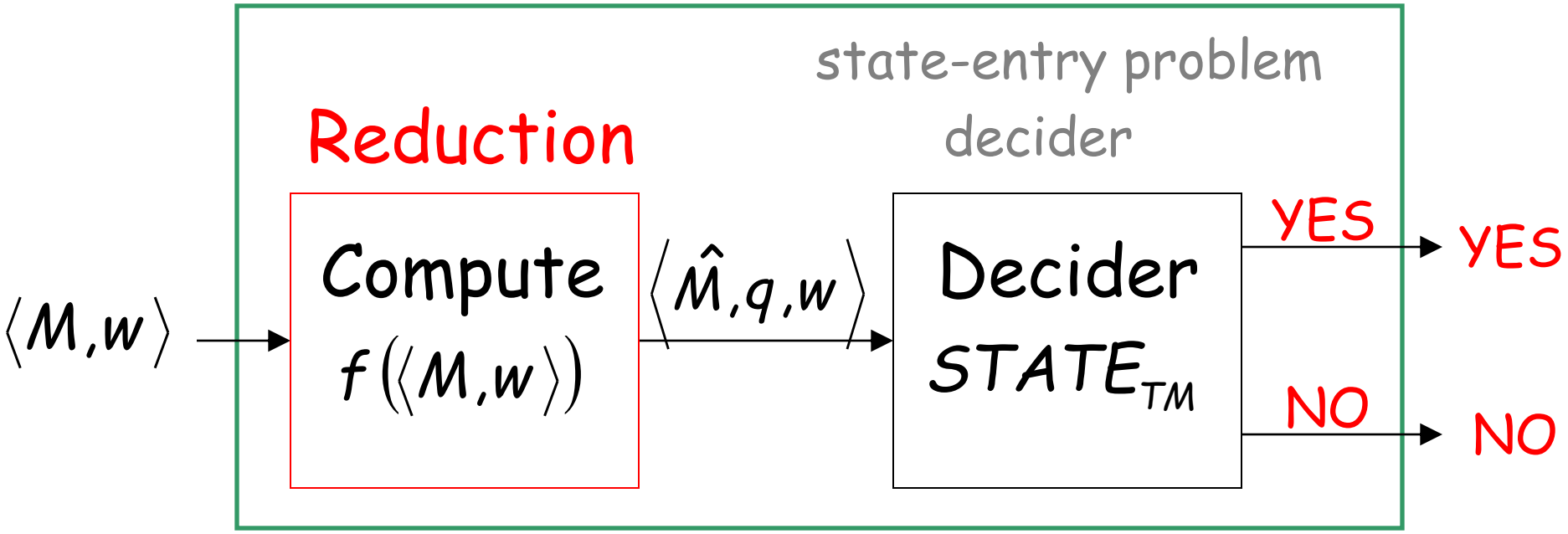
$HALT_{TM}$ (halting problem)

to

$STATE_{TM}$ (state-entry problem)

Halting Problem Decider

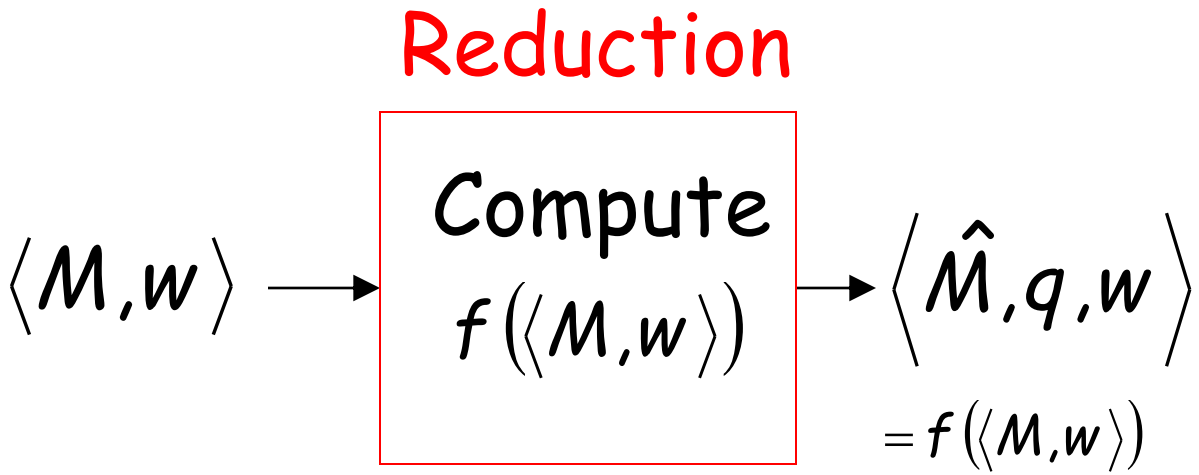
Decider for $HALT_{TM}$



Given the reduction,
if $STATE_{TM}$ is decidable,
then $HALT_{TM}$ is decidable

A contradiction!
since $HALT_{TM}$
is undecidable

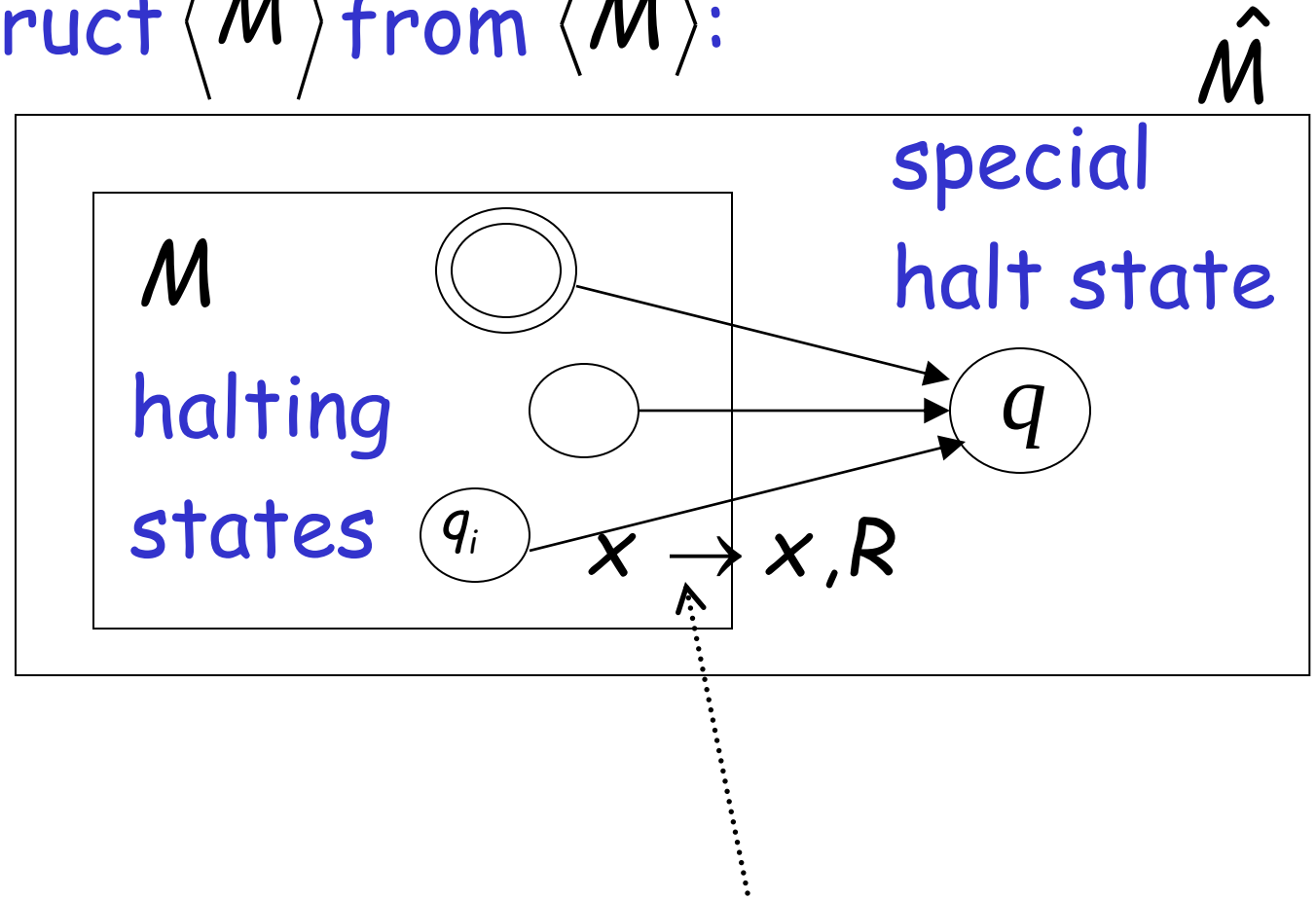
We only need to build the reduction:



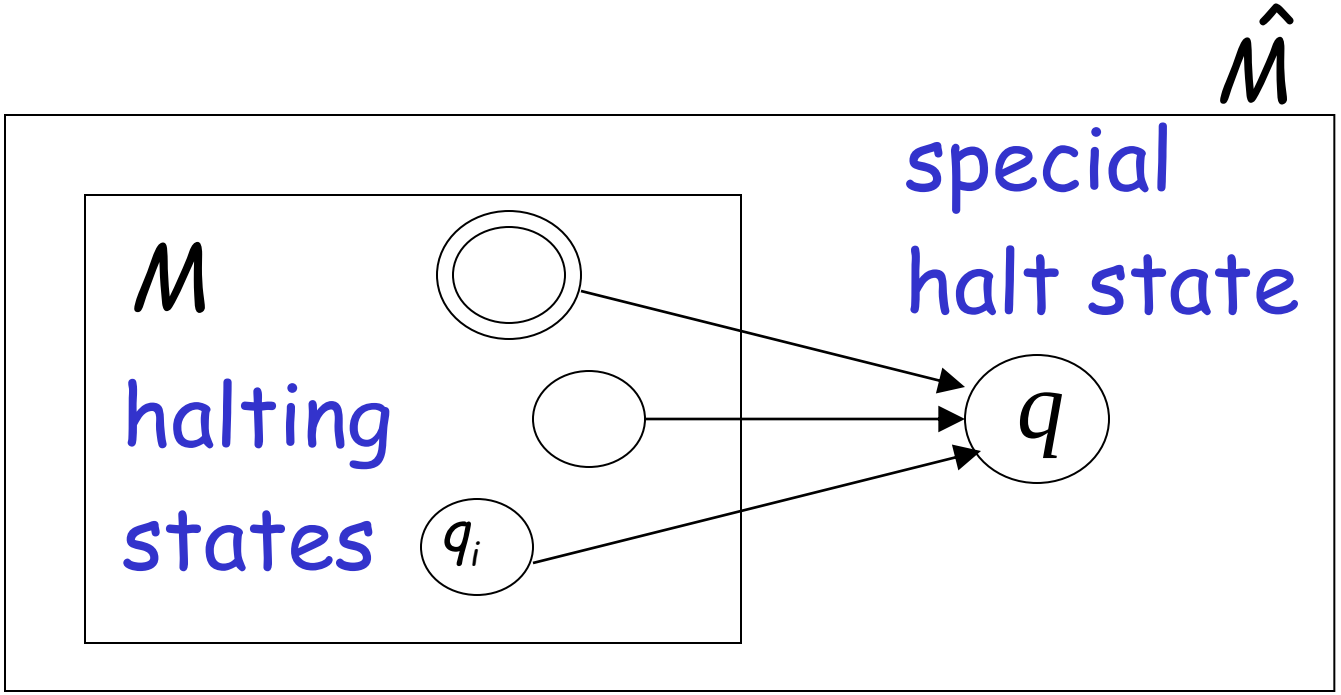
So that:

$$\langle M,w\rangle \in HALT_{TM} \iff \langle \hat{M},w,q\rangle \in STATE_{TM}$$

Construct $\langle \hat{M} \rangle$ from $\langle M \rangle$:

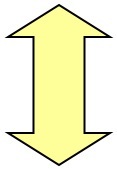


A transition for every unused tape symbol x of q_i



M halts $\Leftrightarrow$ $\hat{M}$ halts on state q

Therefore: M halts on input w



$\hat{M}$ halts on state q on input w

Equivalently:

$$\langle M, w \rangle \in HALT_{TM} \iff \langle \hat{M}, w, q \rangle \in STATE_{TM}$$

END OF PROOF

Blank-tape halting problem

Input: Turing Machine M

Question: Does M halt when started with a blank tape?

Corresponding language:

$$BLANK_{TM} = \{\langle M \rangle : M \text{ is a Turing machine that halts when started on blank tape}\}$$

Theorem: $BLANK_{TM}$ is undecidable

(blank-tape halting problem is unsolvable)

Proof:

Reduce

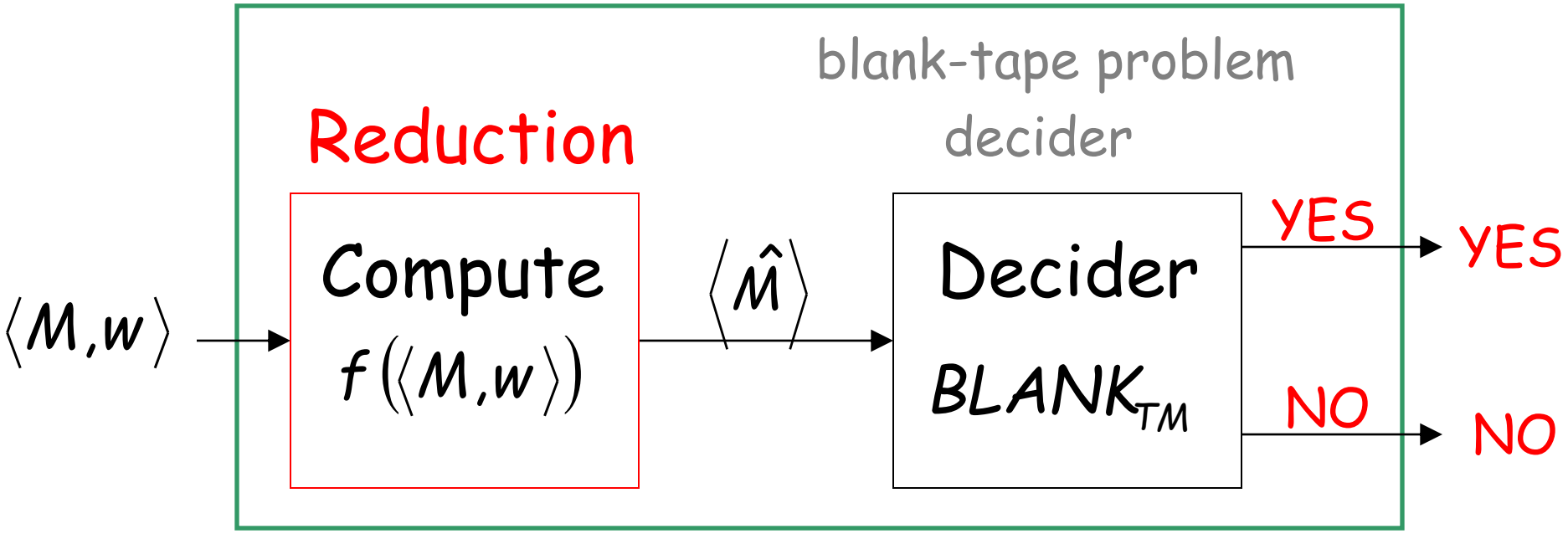
$HALT_{TM}$ (halting problem)

to

$BLANK_{TM}$ (blank-tape problem)

Halting Problem Decider

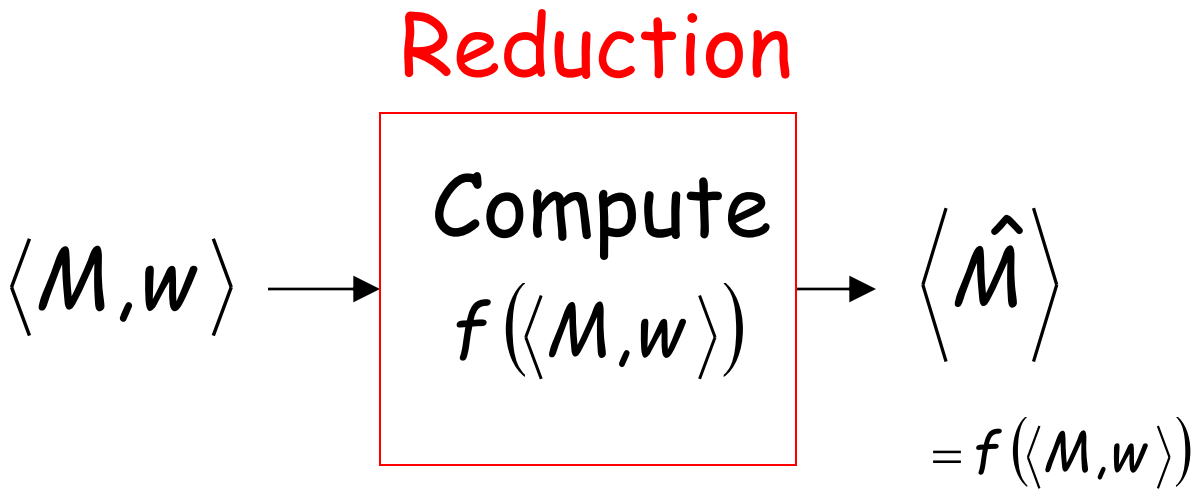
Decider for $HALT_{TM}$



Given the reduction,
If $BLANK_{TM}$ is decidable,
then $HALT_{TM}$ is decidable

A contradiction!
since $HALT_{TM}$
is undecidable

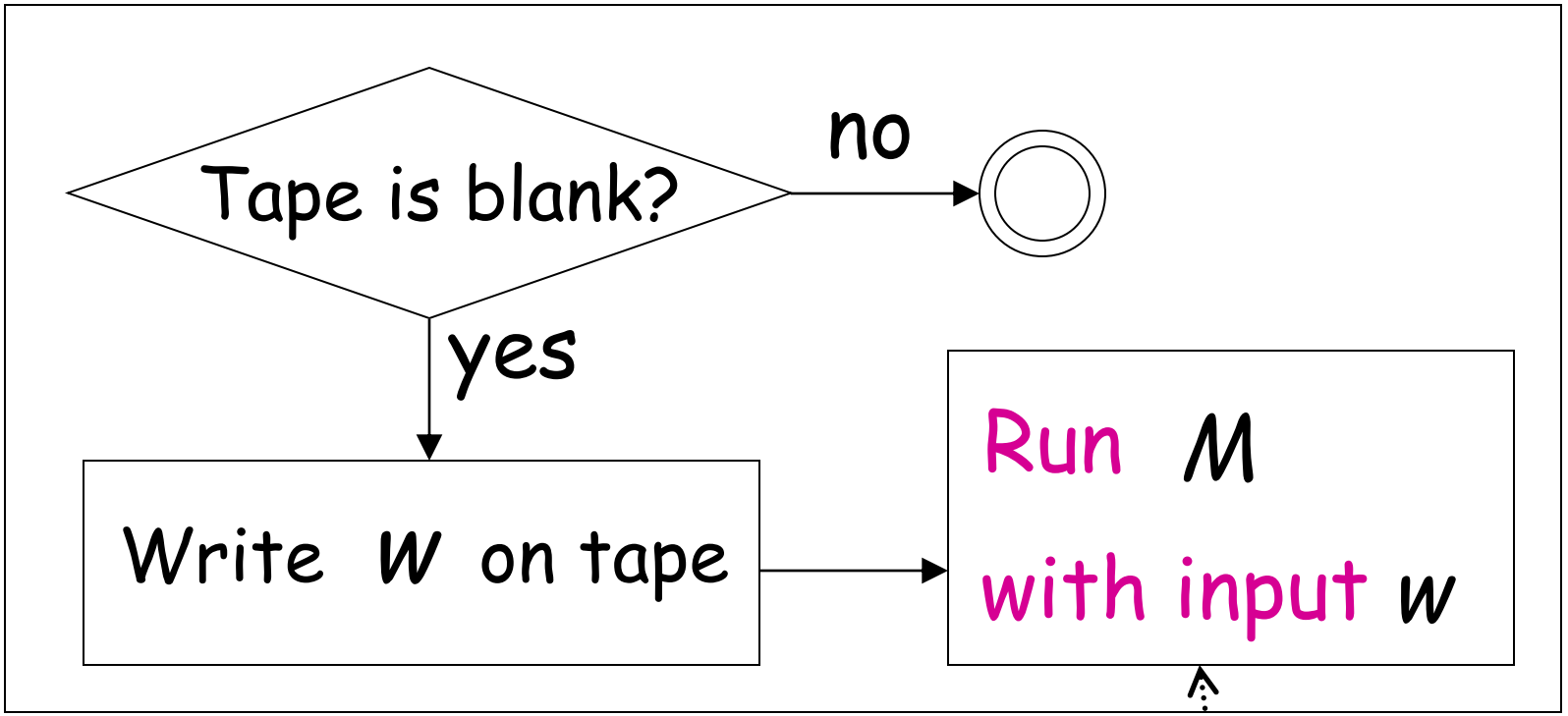
We only need to build the reduction:



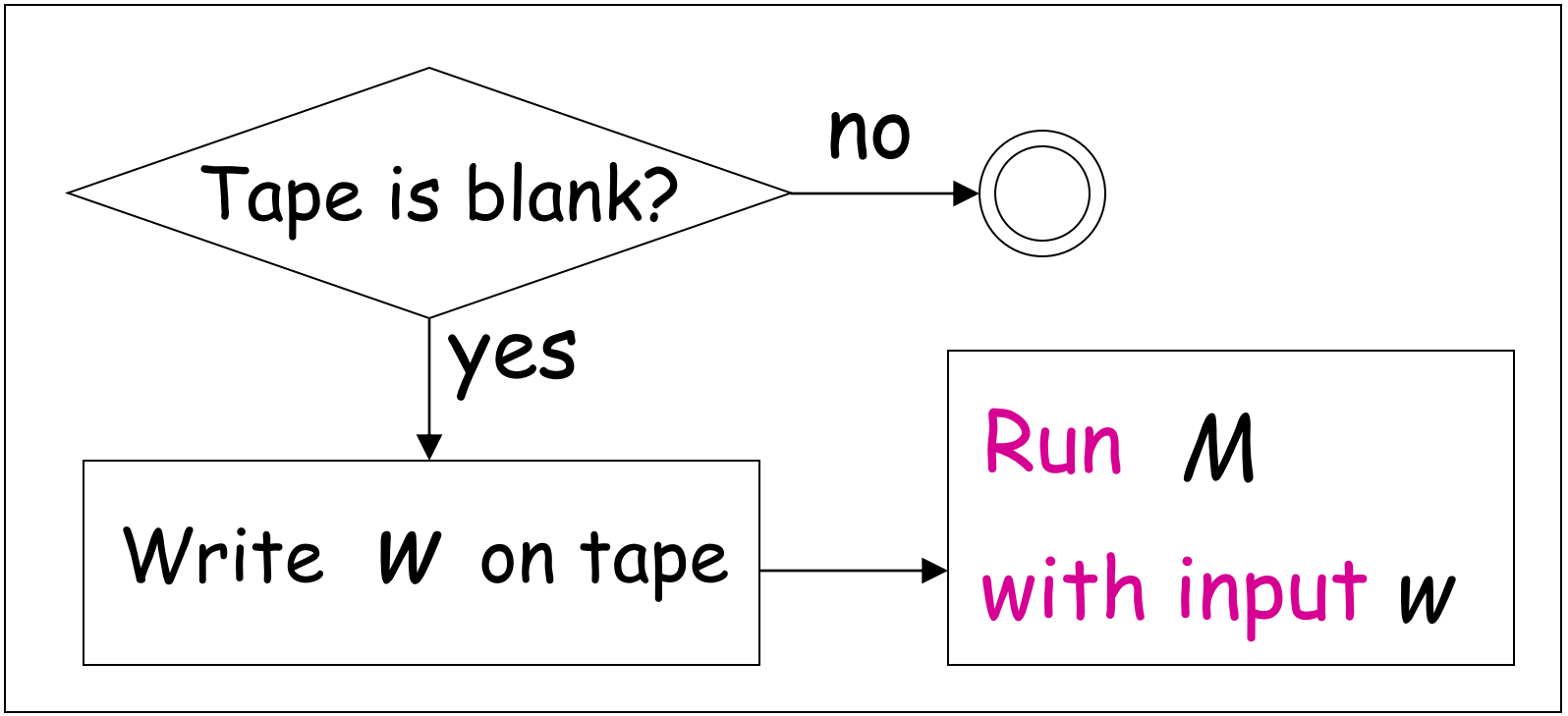
So that:

$$\langle M,w \rangle \in HALT_{TM} \iff \langle \hat{M} \rangle \in BLANK_{TM}$$

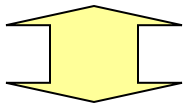
Construct $\langle \hat{M} \rangle$ from $\langle M, w \rangle$:



$\hat{M}$

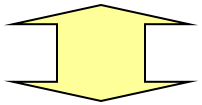


M halts on input w



$\hat{M}$ halts when started on blank tape

M halts on input w



$\hat{M}$ halts when started on blank tape

Equivalently:

$$\langle M, w \rangle \in HALT_{TM} \iff \langle \hat{M} \rangle \in BLANK_{TM}$$

END OF PROOF

Theorem (version 2):

If: a: Language A is reduced to $\overline{B}$

b: Language A is undecidable

Then: B is undecidable

Proof:

Suppose B is decidable

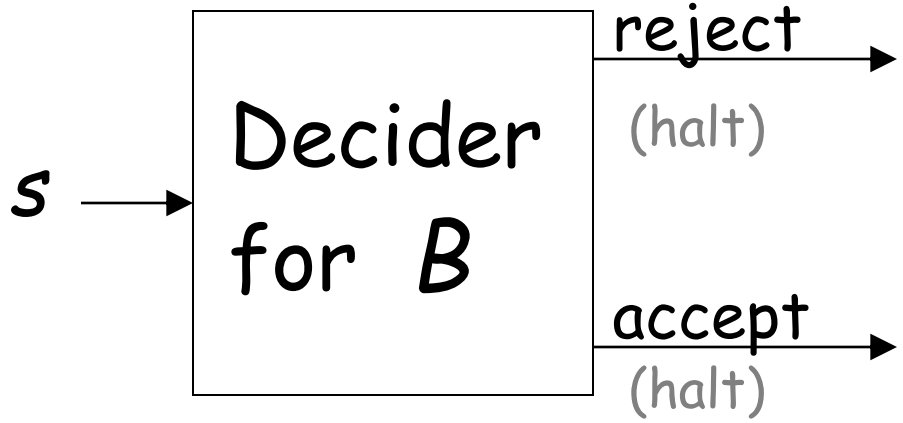
Then $\overline{B}$ is decidable

Using the decider for $\overline{B}$

build the decider for A

Contradiction!

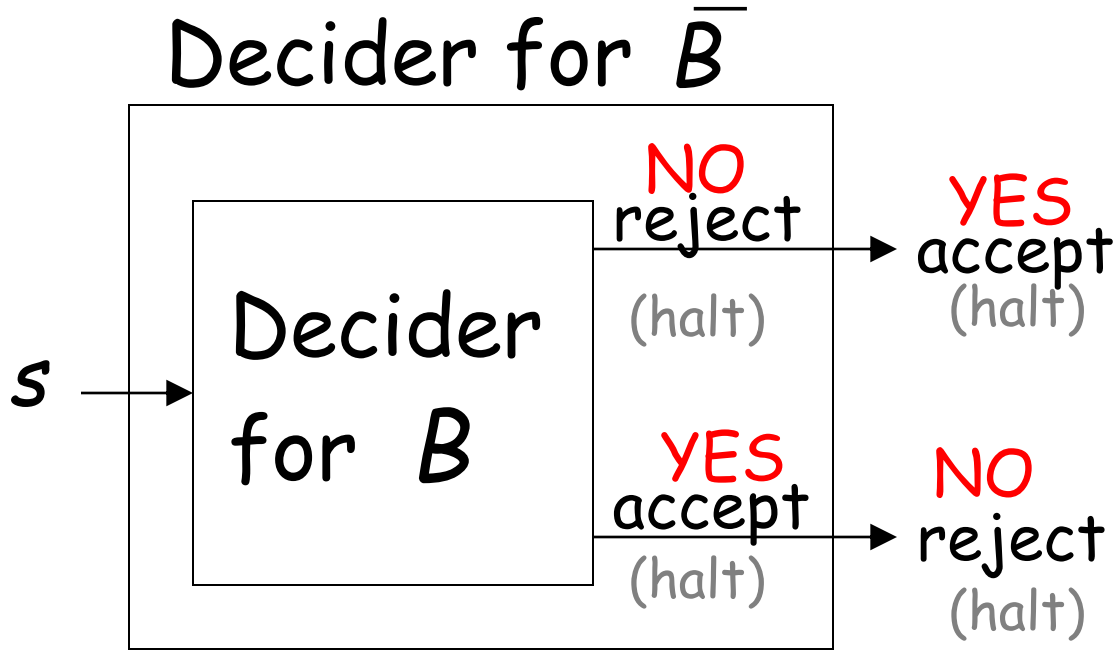
Suppose B is decidable



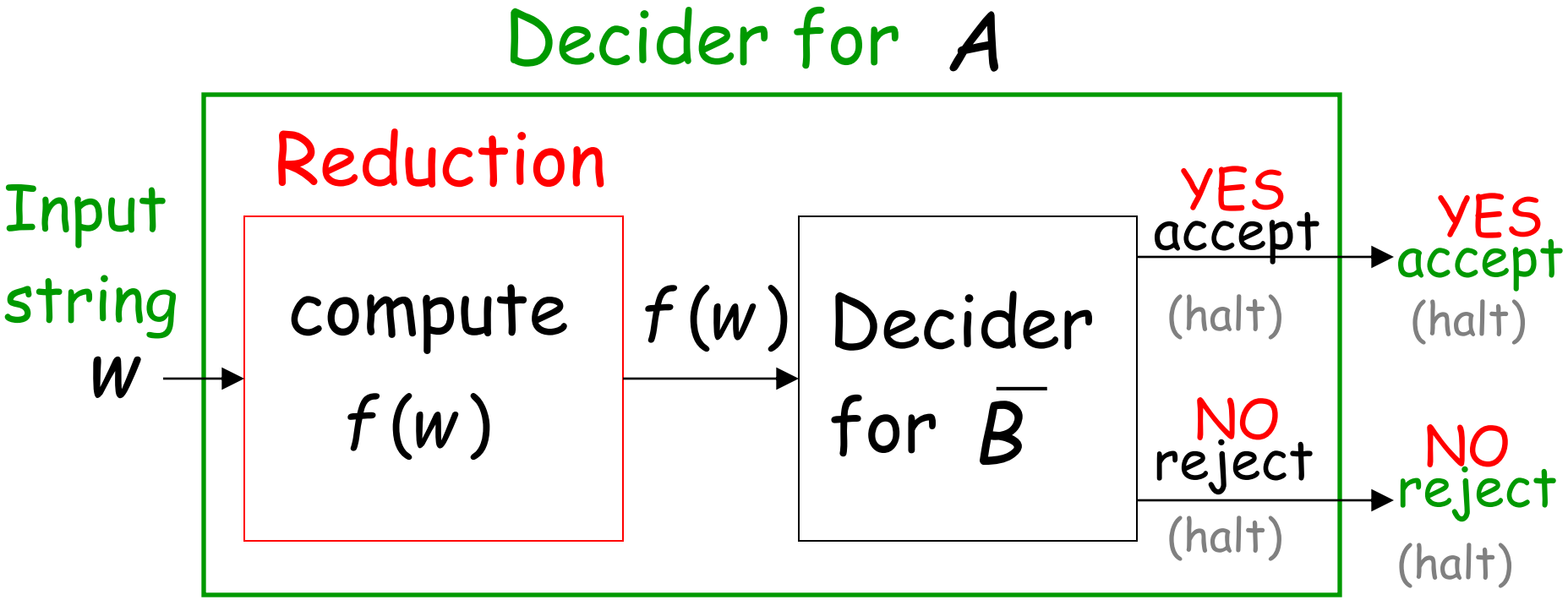
Suppose B is decidable

Then $\overline{B}$ is decidable

(we have proven this in previous class)



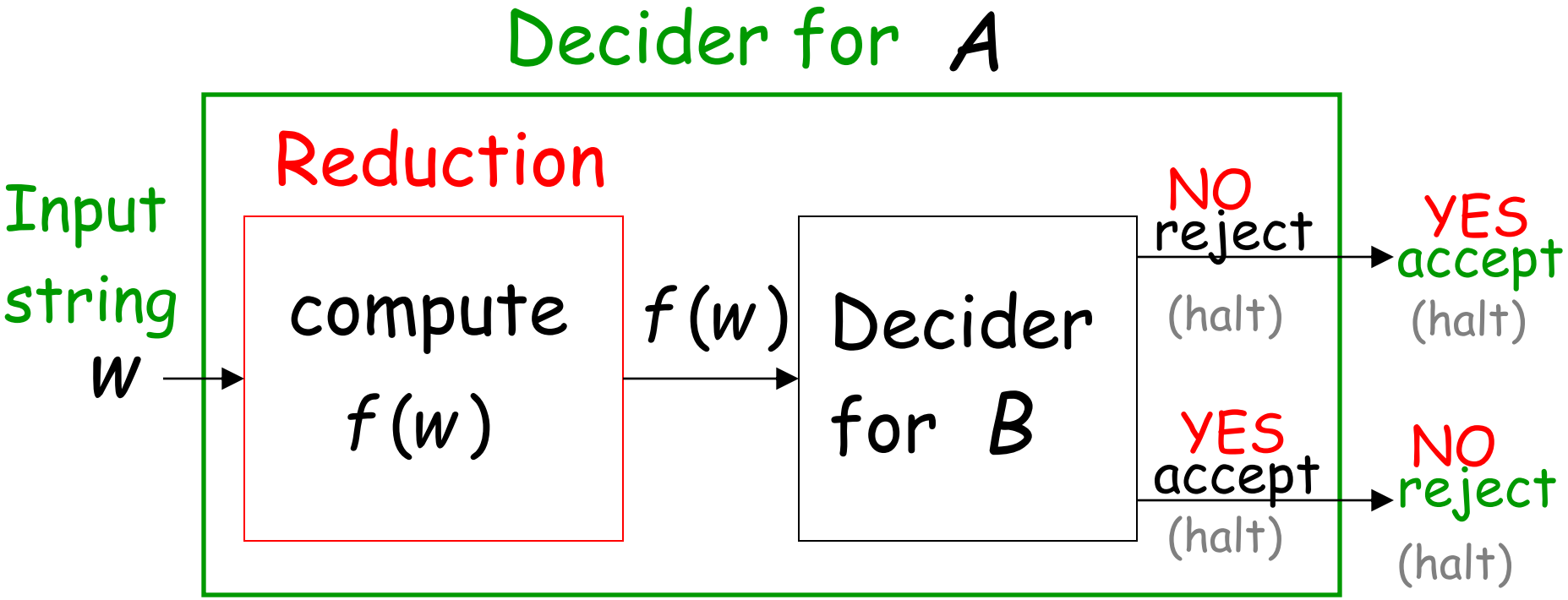
If $\overline{B}$ is decidable then we can build:



$$w \in A \Leftrightarrow f(w) \in \overline{B}$$

CONTRADICTION!

Alternatively:



$$w \in A \Leftrightarrow f(w) \notin B$$

CONTRADICTION!

END OF PROOF

Observation:

In order to prove
that some language B is undecidable
we only need to reduce some
known undecidable language A
to B (theorem version 1)
or to $\overline{B}$ (theorem version 2)

Undecidable Problems for Turing Recognizable languages

Let L be a Turing-acceptable language

- L is empty?
- L is regular?
- L has size 2?

All these are undecidable problems

Let L be a Turing-acceptable language

- L is empty?
- L is regular?
- L has size 2?

Empty language problem

Input: Turing Machine M

Question: Is $L(M)$ empty? $L(M) = \emptyset$?

Corresponding language:

$$EMPTY_{TM} = \{\langle M \rangle : M \text{ is a Turing machine that accepts the empty language } \emptyset\}$$

Theorem: $EMPTY_{TM}$ is undecidable

(empty-language problem is unsolvable)

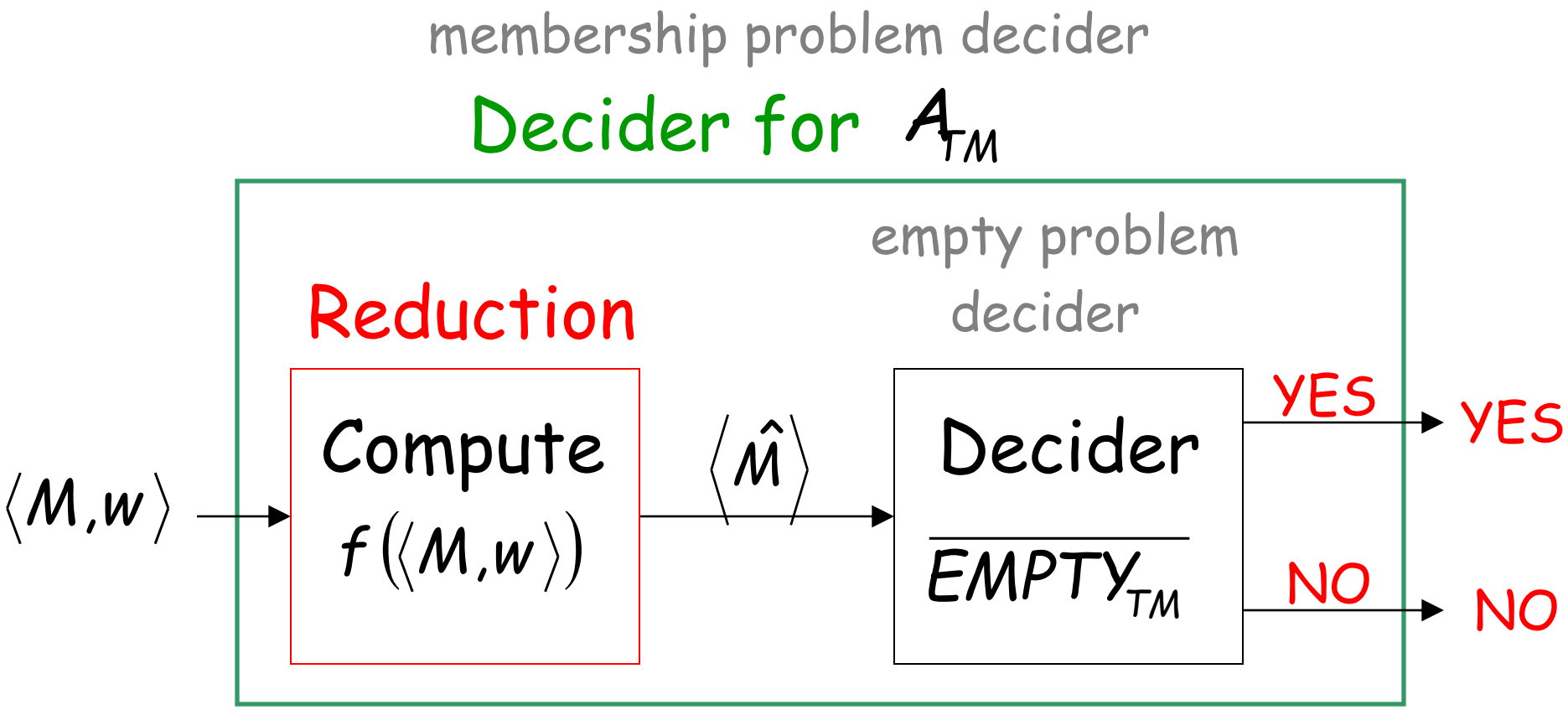
Proof:

Reduce

A_{TM} (membership problem)

to

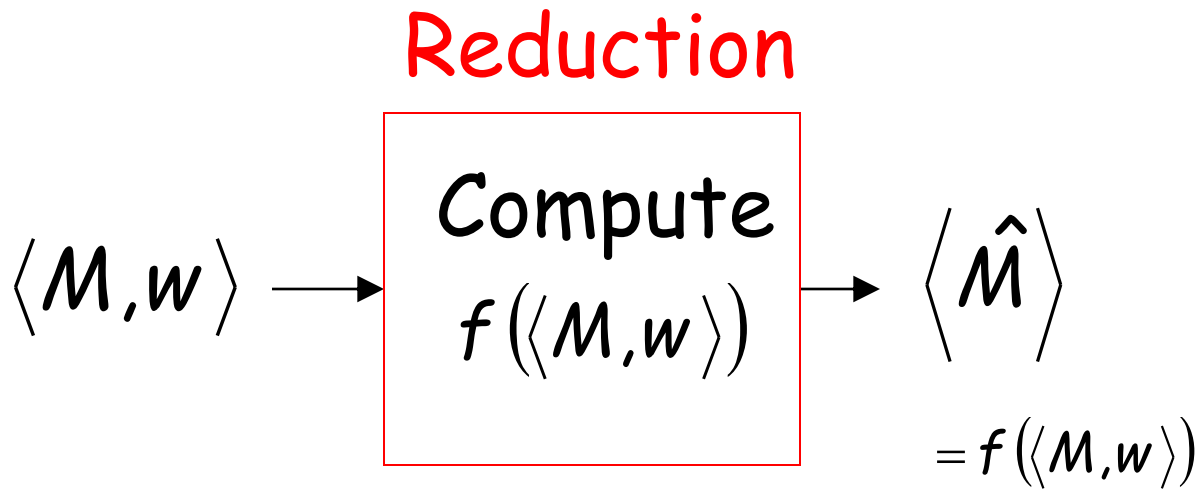
$\overline{EMPTY_{TM}}$ (empty language problem)



Given the reduction,
if $\overline{EMPTY_{TM}}$ is decidable,
then A_{TM} is decidable

A contradiction!
since A_{TM}
is undecidable

We only need to build the reduction:

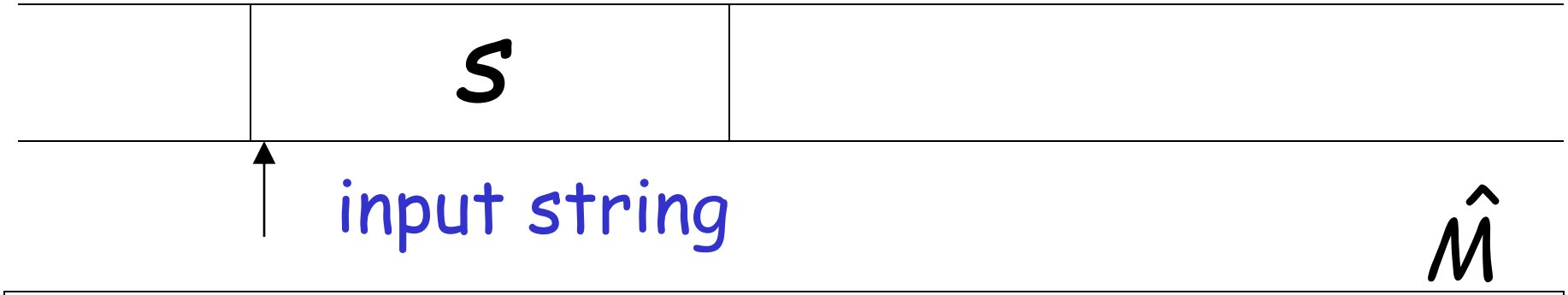


So that:

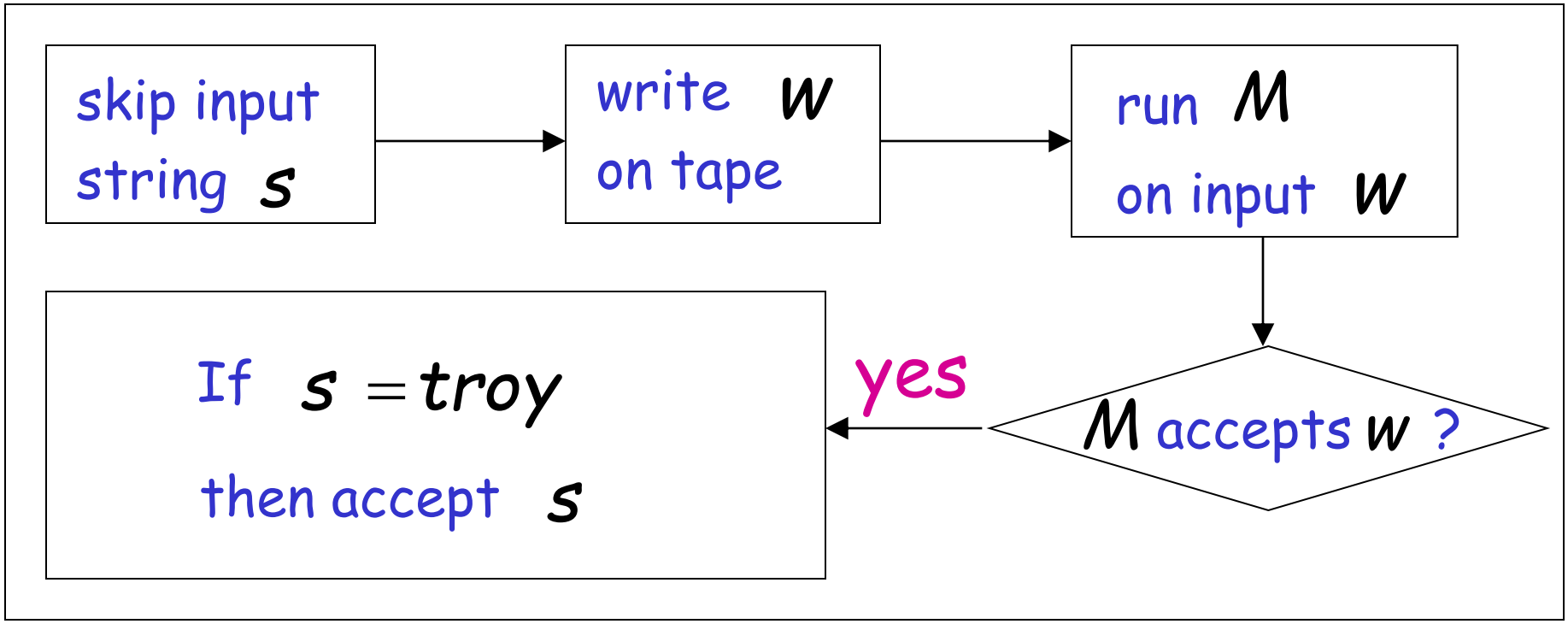
$$\langle M,w \rangle \in AT_{TM} \iff \langle \hat{M} \rangle \in \overline{EMPTY_{TM}}$$

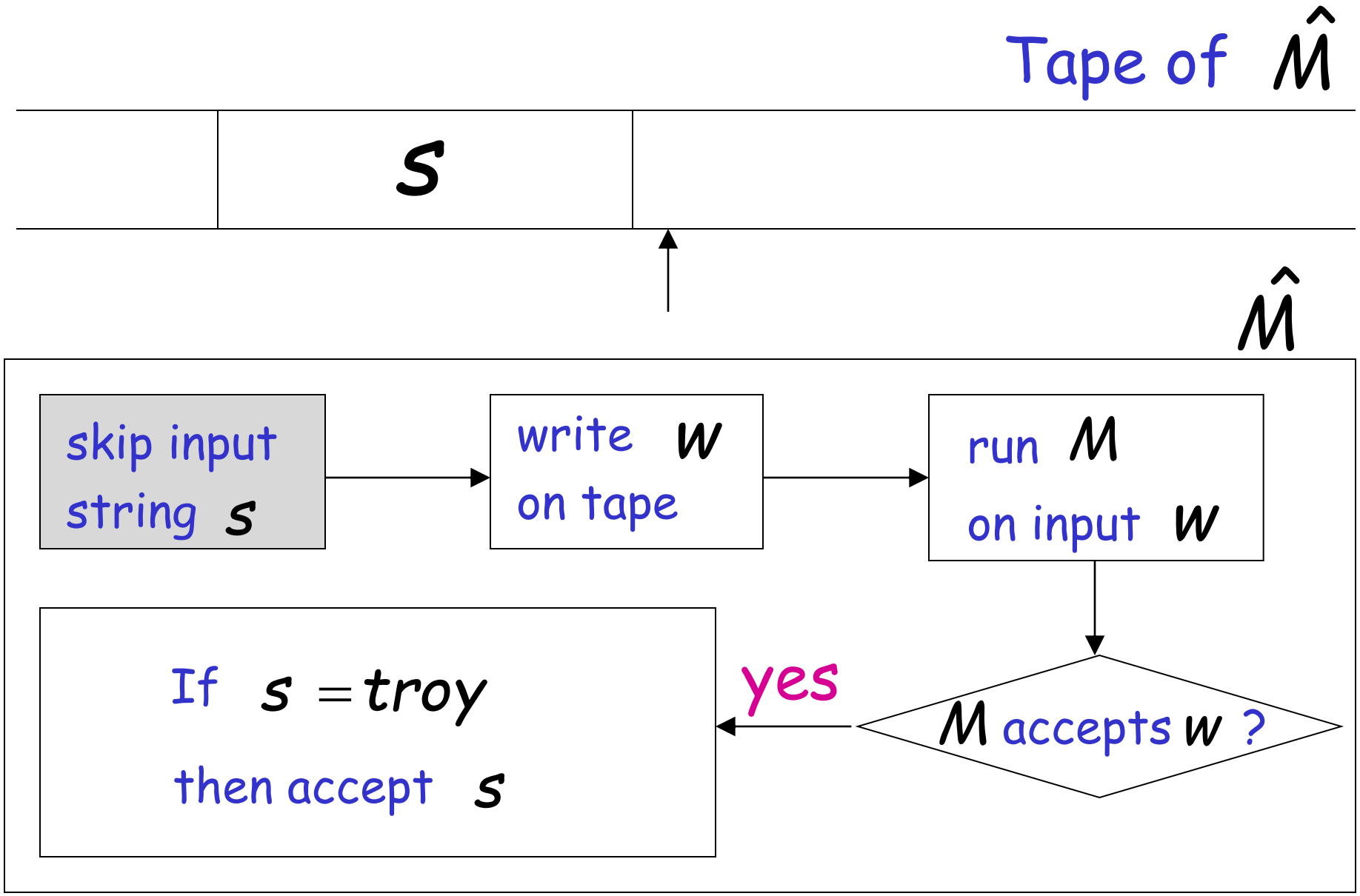
Construct $\langle \hat{M} \rangle$ from $\langle M, w \rangle$:

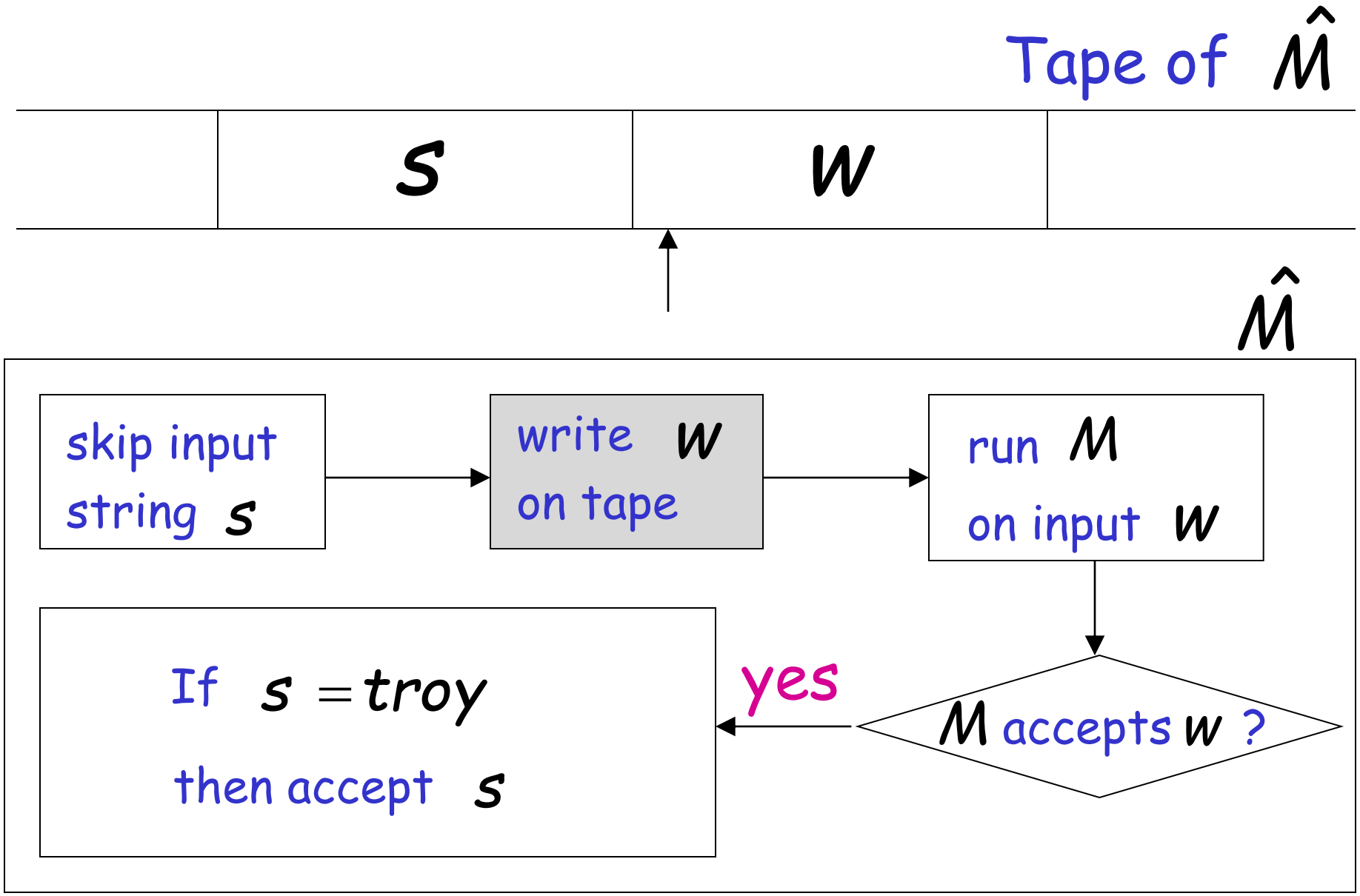
Tape of $\hat{M}$



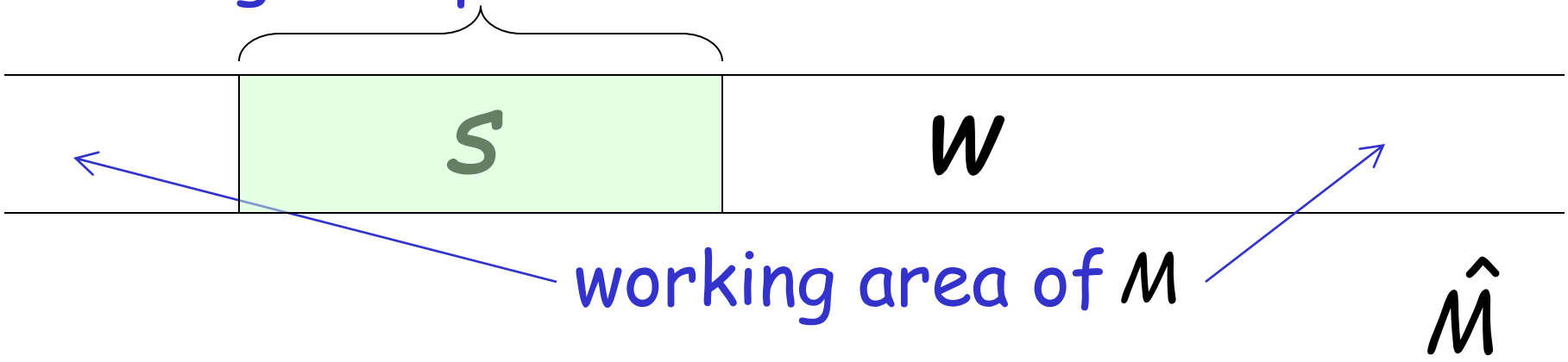
$\hat{M}$



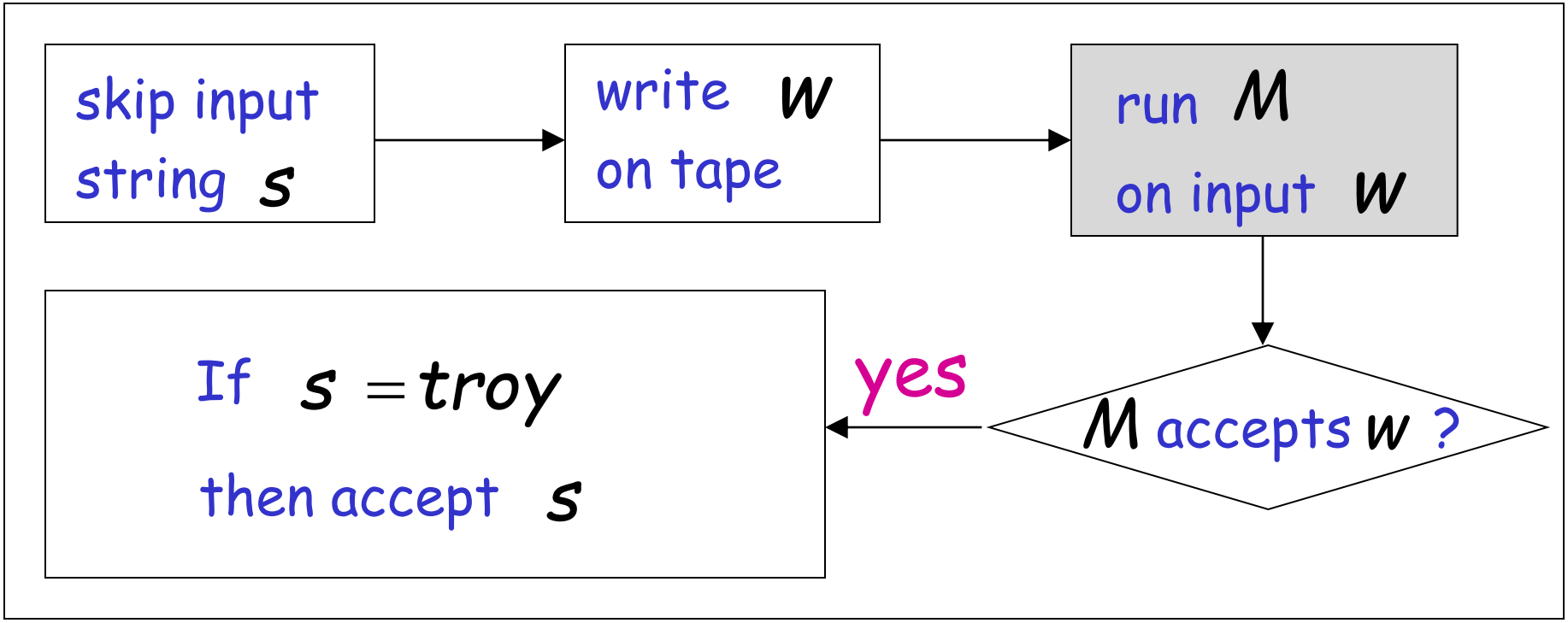

Tape of $\hat{M}$
s
$\hat{M}$
skip input string s
write w on tape
run M on input w
M accepts w ?
yes
If $s = troy$
then accept s


Tape of $\hat{M}$
s
w
$\hat{M}$
skip input string s
write w on tape
run M on input w
M accepts w ?
yes
If s = troy
then accept s

During this phase this area is not touched

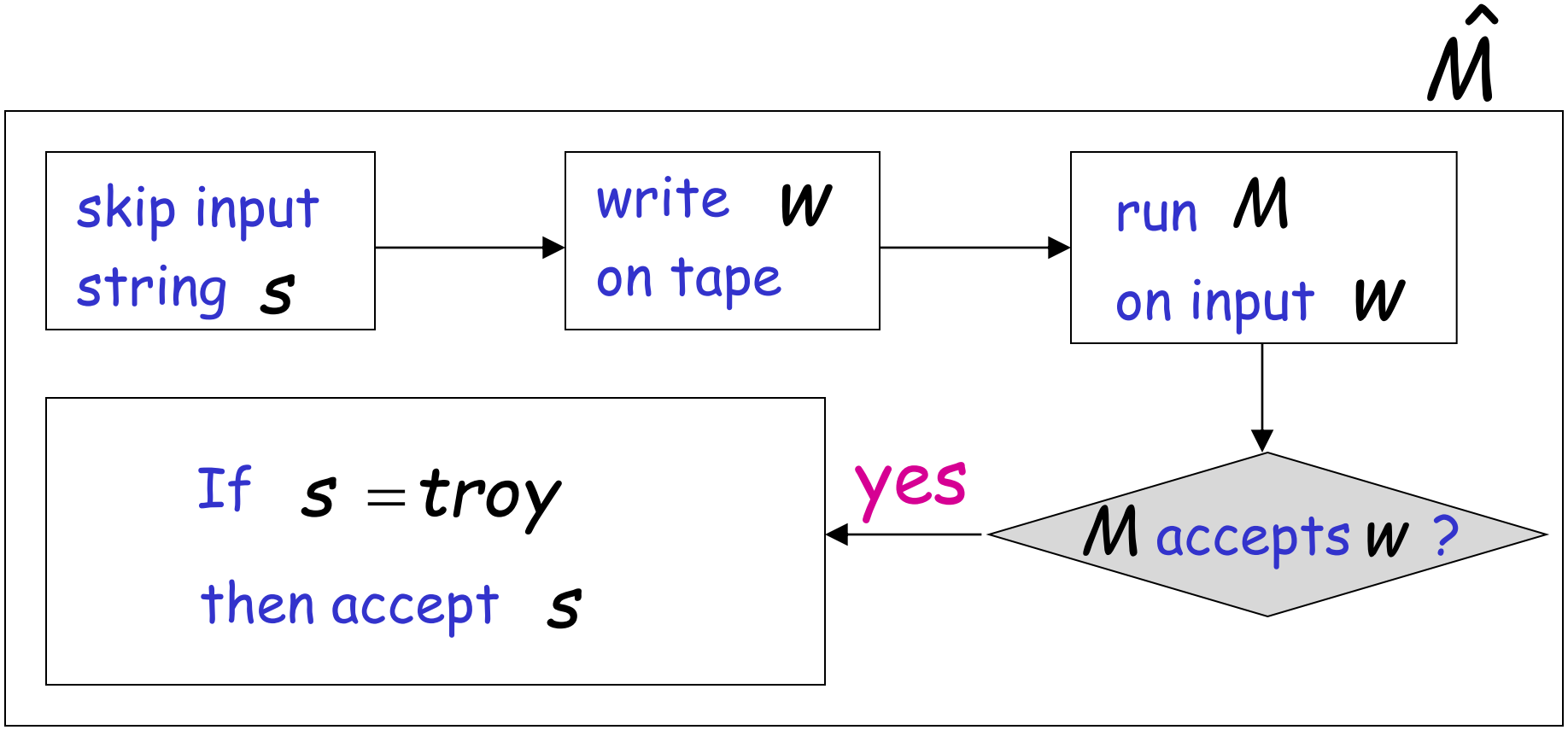


$\hat{M}$

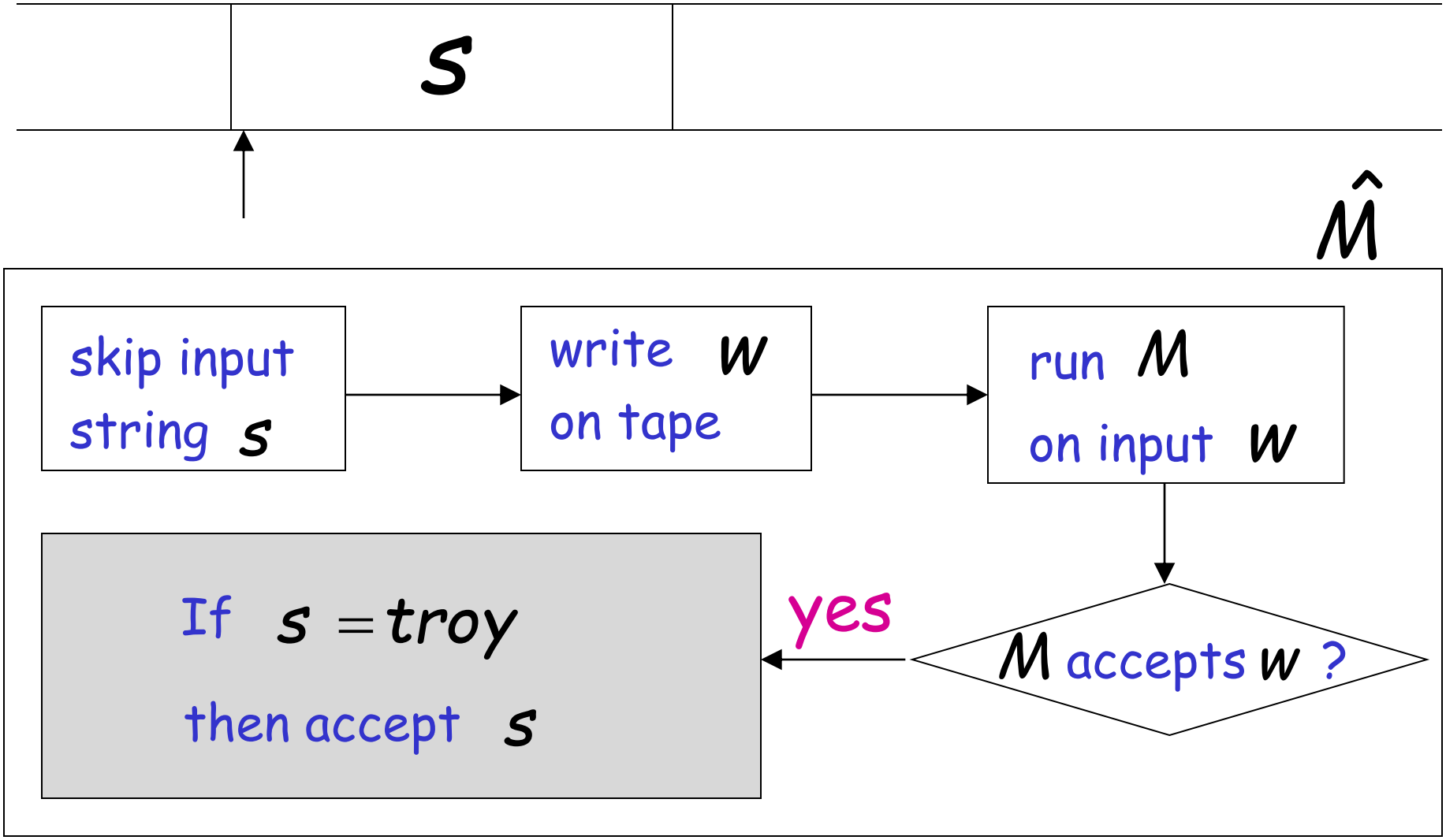


Simply check if M entered an accept state

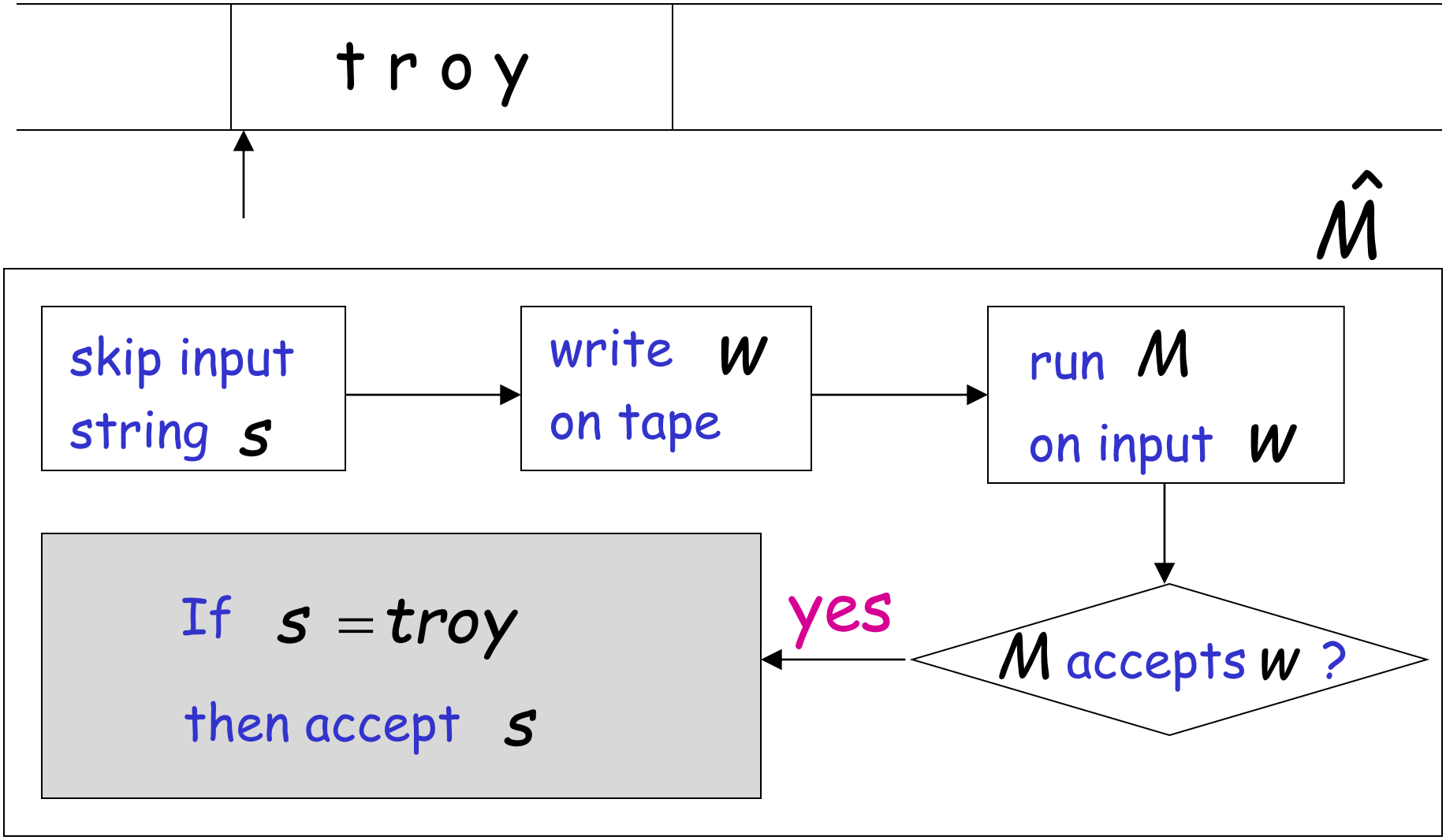
	s	altered w



Now check input string

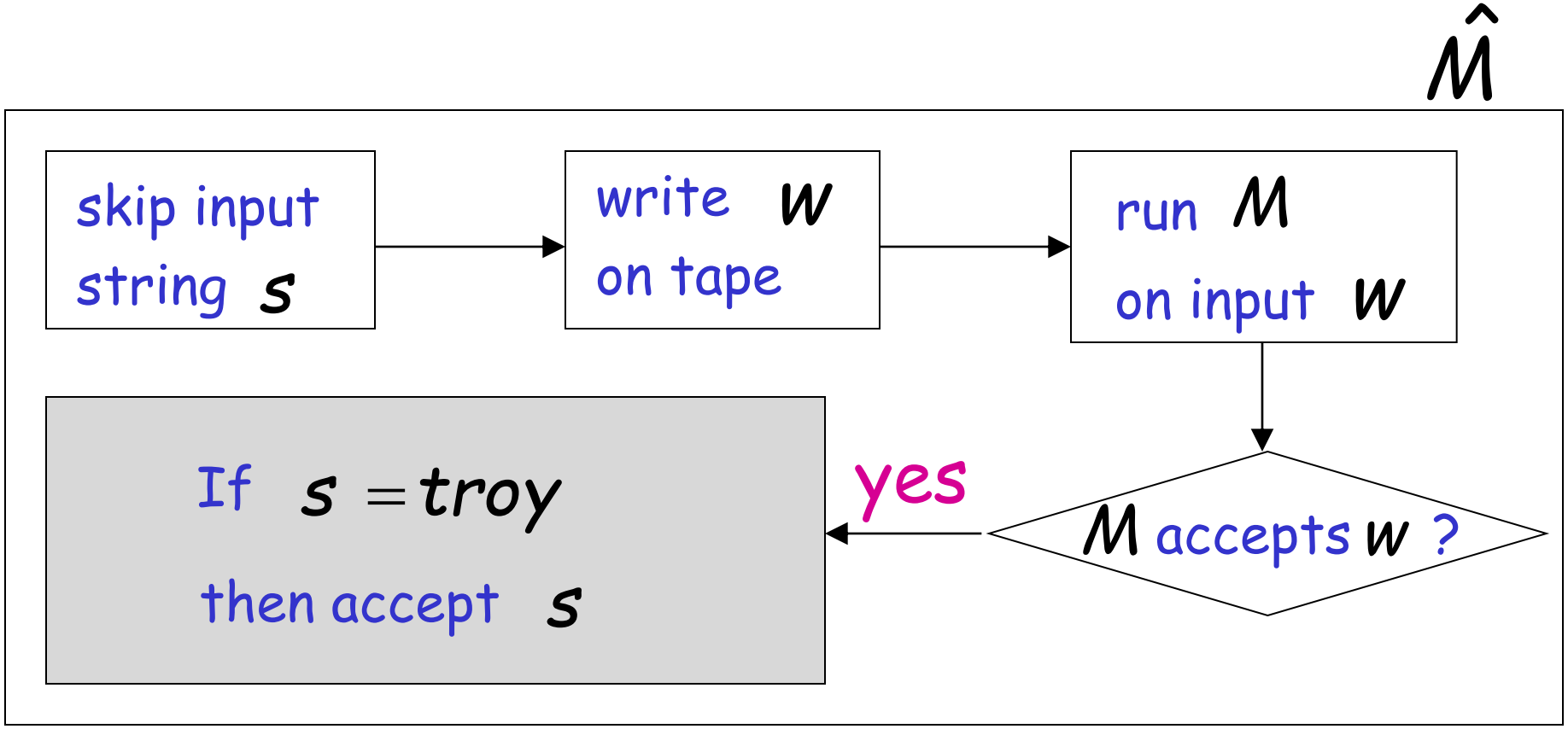


The only possible accepted string



M accepts w ⇒ $L(\hat{M}) = \{troy\} \neq \emptyset$

M does not accept w ⇒ $L(\hat{M}) = \emptyset$



Therefore:

M accepts w $\Longleftrightarrow$ $L(\hat{M}) \neq \emptyset$

Equivalently:

$\langle M, w \rangle \in AT_{TM}$ $\Longleftrightarrow$ $\langle \hat{M} \rangle \in \overline{EMPTY_{TM}}$

END OF PROOF

Let L be a Turing-acceptable language

- L is empty?
- L is regular?
- L has size 2?

Regular language problem

Input: Turing Machine M

Question: Is $L(M)$ a regular language?

Corresponding language:

$$REGULAR_{TM} = \{\langle M \rangle : M \text{ is a Turing machine that accepts a regular language}\}$$

Theorem: $REGULAR_{TM}$ is undecidable

(regular language problem is unsolvable)

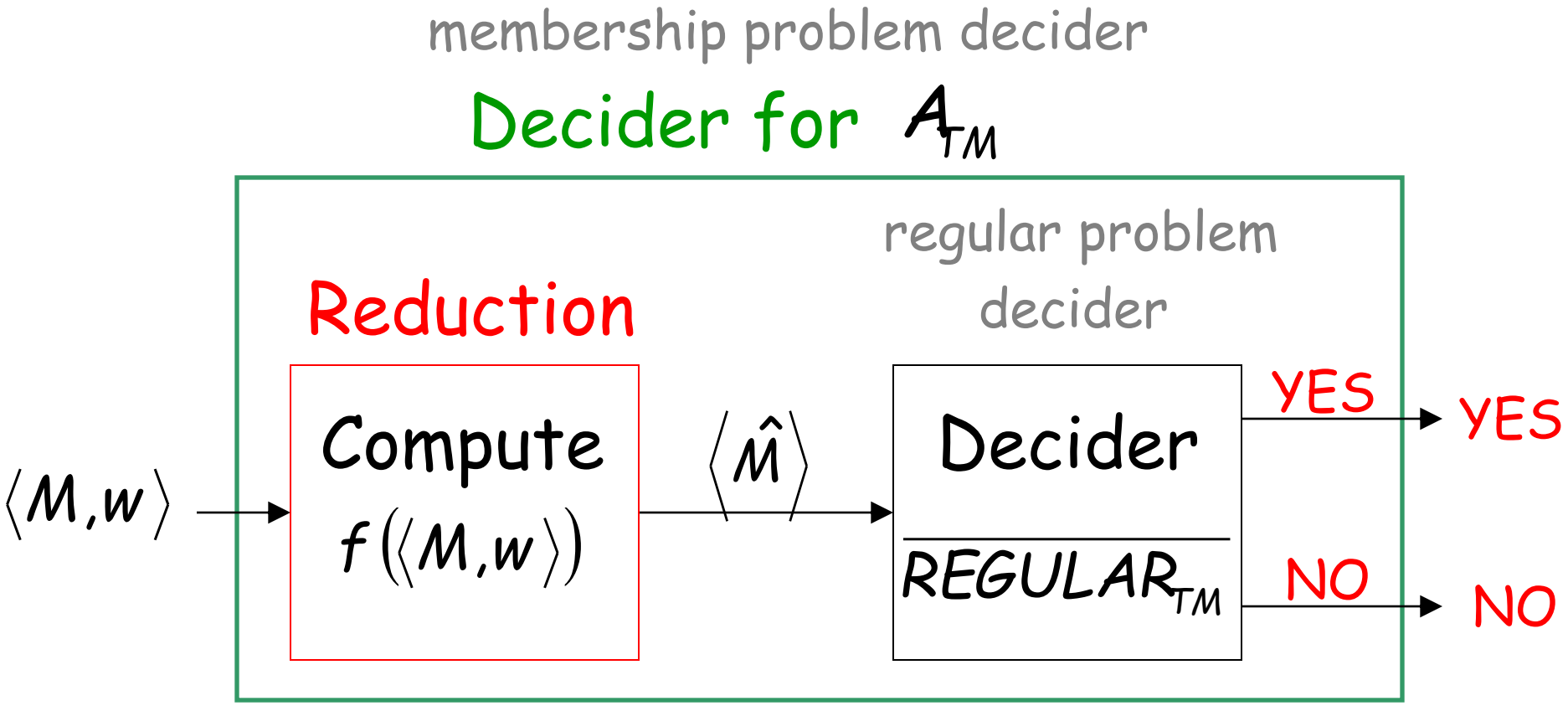
Proof:

Reduce

A_{TM} (membership problem)

to

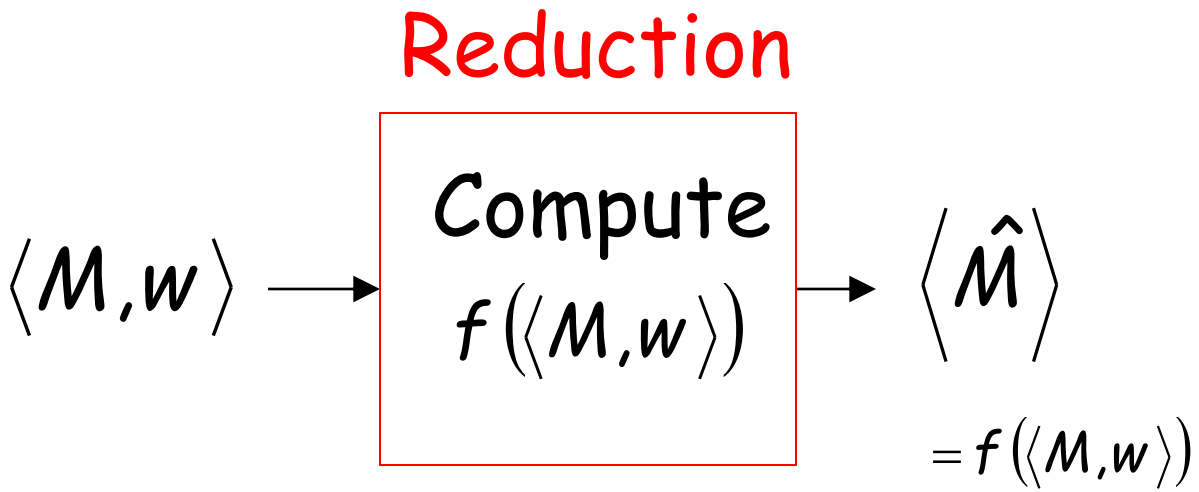
$\overline{REGULAR_{TM}}$ (regular language problem)



Given the reduction,
If $\overline{REGULAR_{TM}}$ is decidable,
then A_{TM} is decidable

A contradiction!
since A_{TM}
is undecidable

We only need to build the reduction:

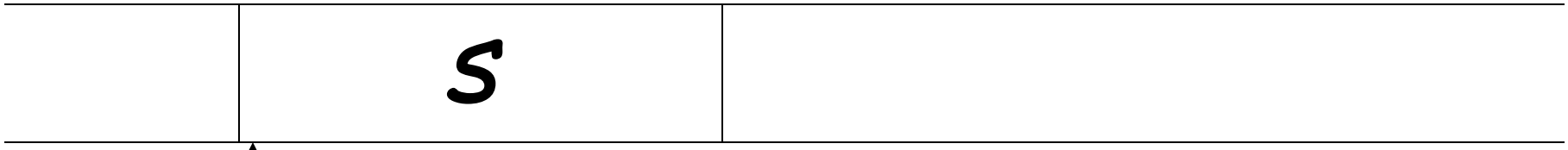


So that:

$$\langle M,w \rangle \in AT_{TM} \iff \langle \hat{M} \rangle \in \overline{REGULAR_{TM}}$$

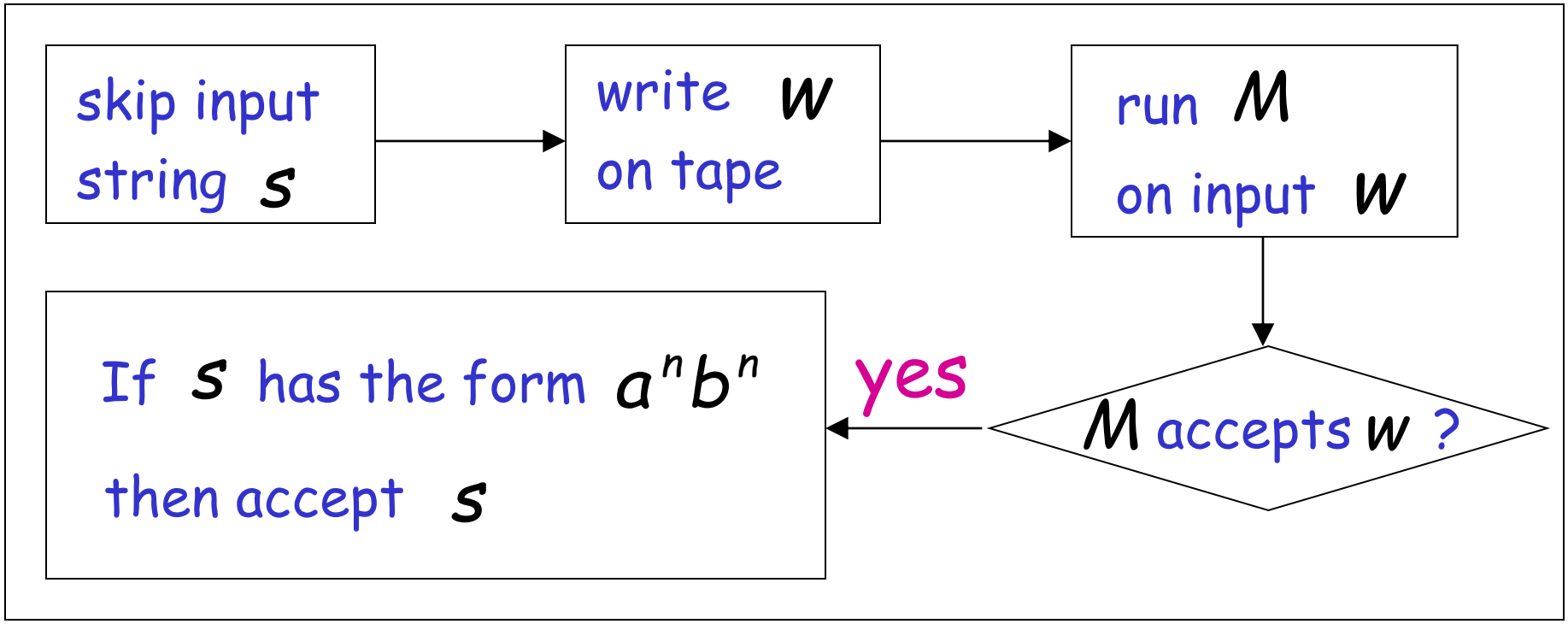
Construct $\langle \hat{M} \rangle$ from $\langle M, w \rangle$:

Tape of $\hat{M}$



input string

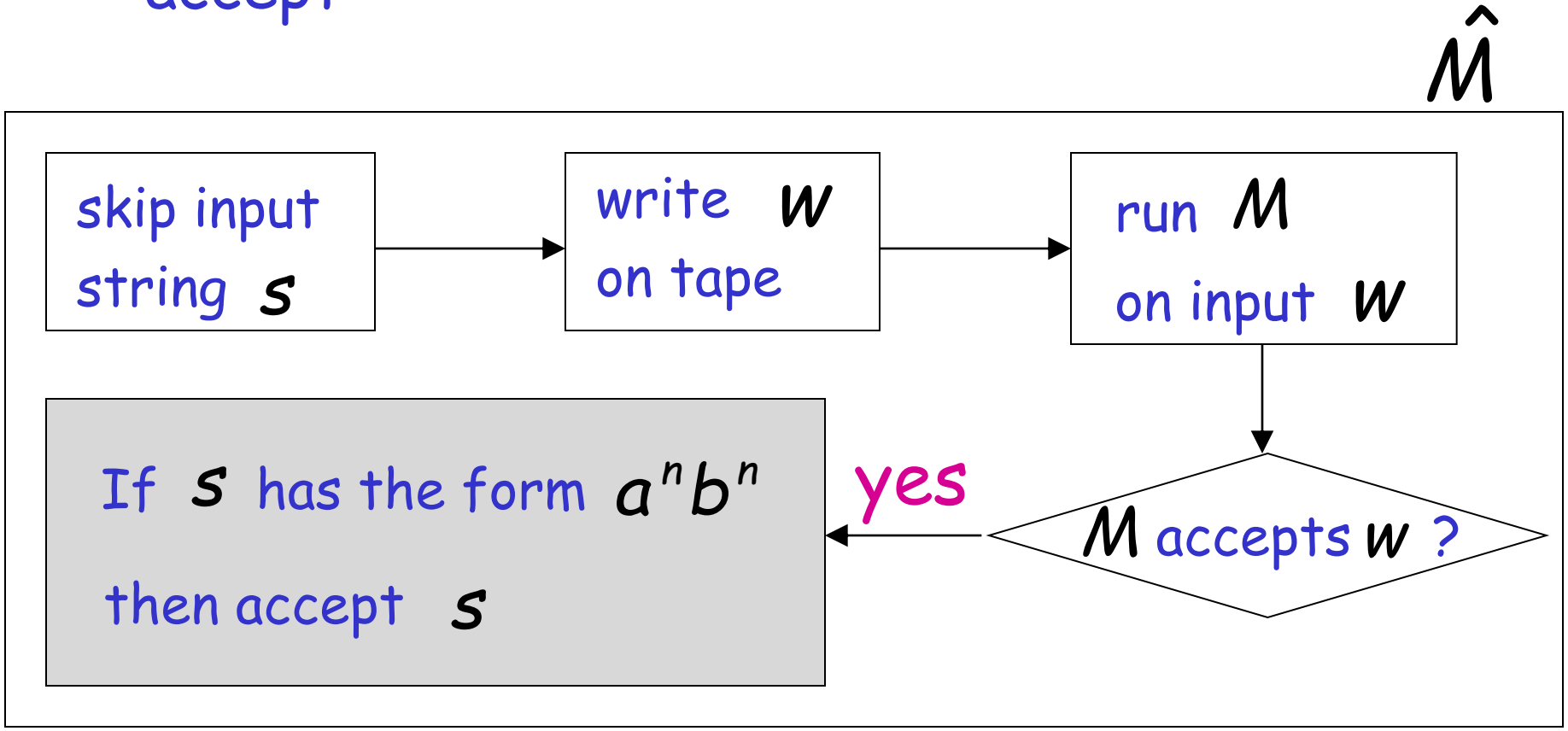
$\hat{M}$



not regular

M accepts w $\Rightarrow$ $L(\hat{M}) = \{a^n b^n : n \geq 0\}$

M does not accept w $\Rightarrow$ $L(\hat{M}) = \emptyset$ regular



Therefore:

M accepts w $\Longleftrightarrow$ $L(\hat{M})$ is not regular

Equivalently:

$$\langle M, w \rangle \in AT_{TM} \Longleftrightarrow \langle \hat{M} \rangle \in \overline{REGULAR_{TM}}$$

END OF PROOF

Let L be a Turing-acceptable language

- L is empty?
- L is regular?
- L has size 2?

Size2 language problem

Input: Turing Machine M

Question: Does $L(M)$ have size 2? $|L(M)|=2$?

(accepts exactly two strings?)

Corresponding language:

$$SIZE2_{TM} = \{\langle M \rangle : M \text{ is a Turing machine that accepts exactly two strings}\}$$

Theorem: $SIZE2_{TM}$ is undecidable

(regular language problem is unsolvable)

Proof:

Reduce

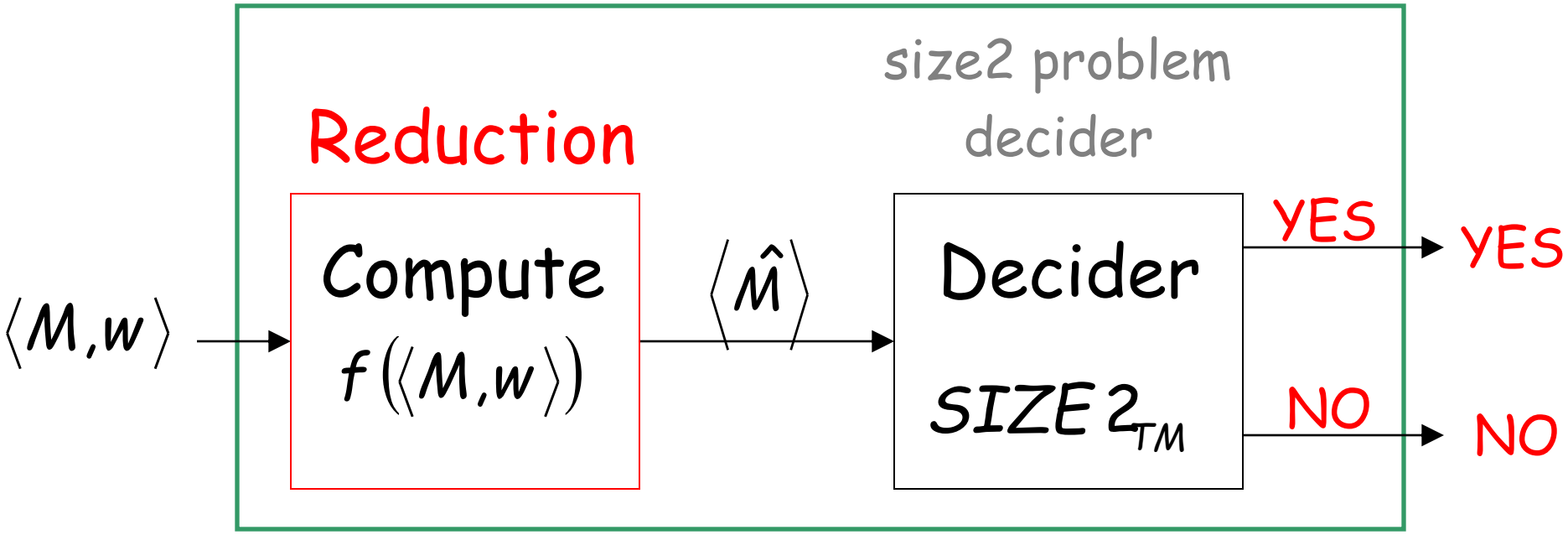
A_{TM} (membership problem)

to

$SIZE2_{TM}$ (size 2 language problem)

membership problem decider

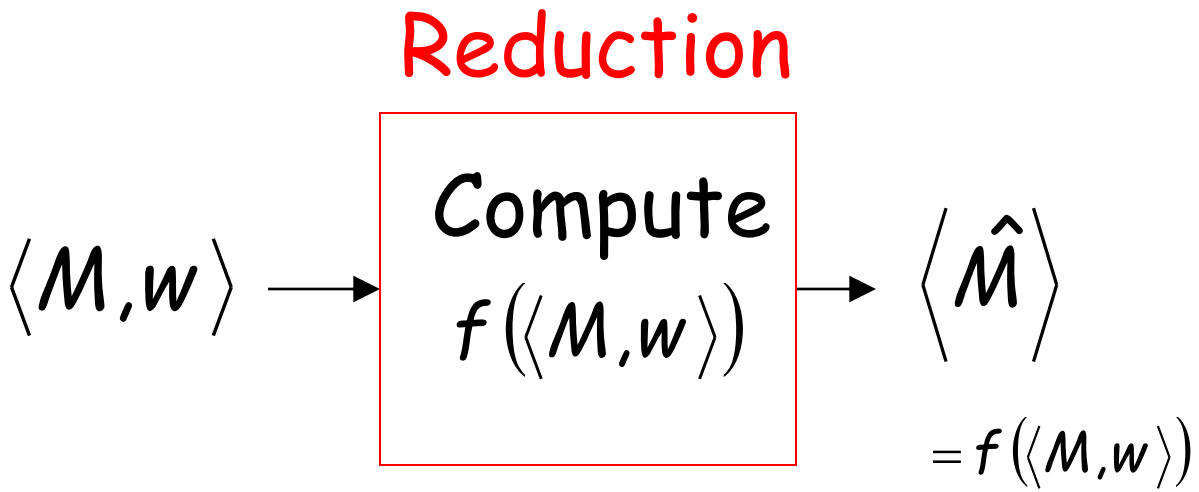
Decider for A_{TM}



Given the reduction,
If $SIZE2_{TM}$ is decidable,
then A_{TM} is decidable

A contradiction!
since A_{TM}
is undecidable

We only need to build the reduction:

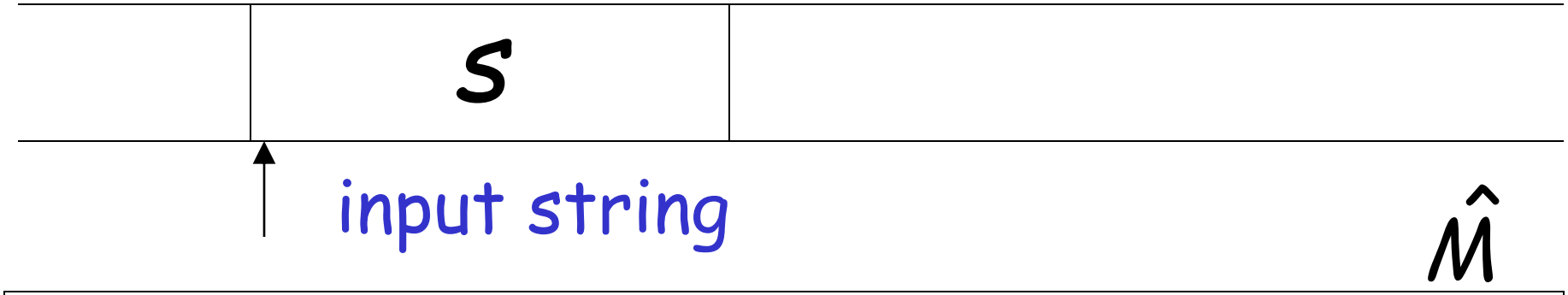


So that:

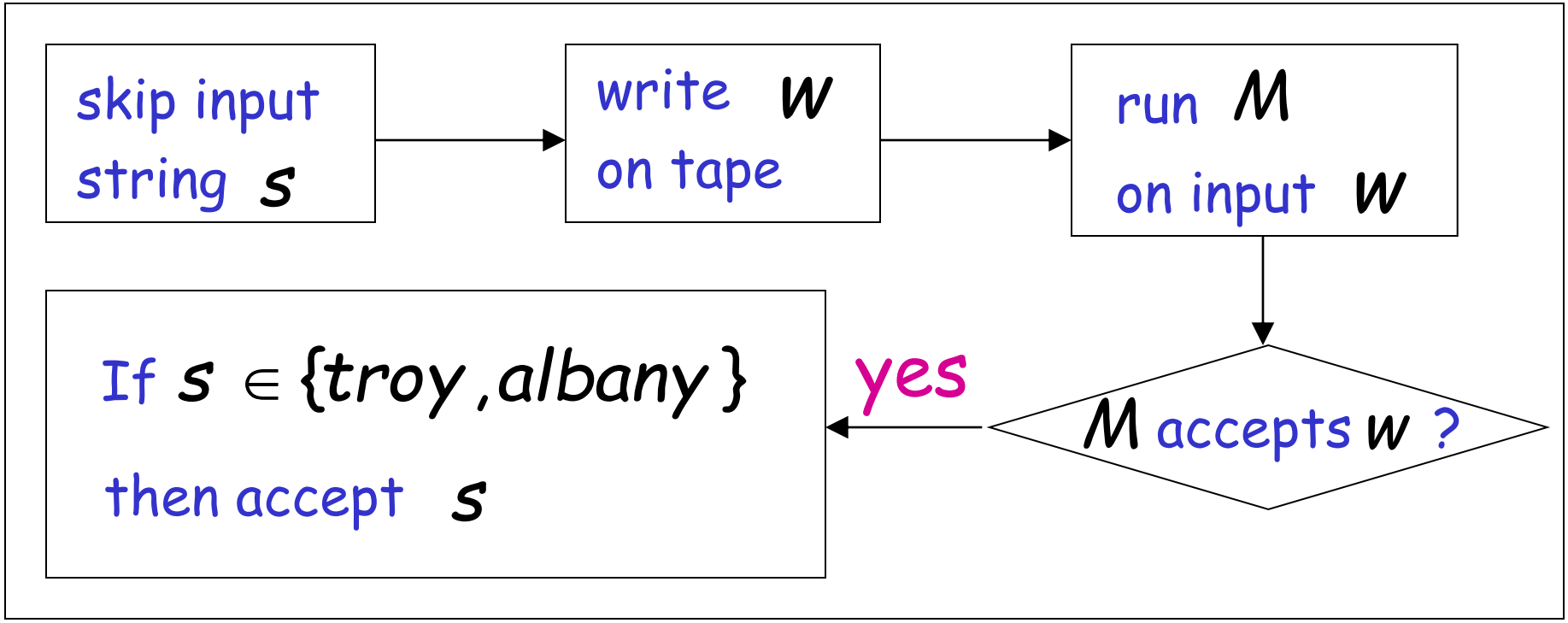
$$\langle M,w \rangle \in AT_{TM} \iff \langle \hat{M} \rangle \in SIZE2_{TM}$$

Construct $\langle \hat{M} \rangle$ from $\langle M, w \rangle$:

Tape of $\hat{M}$



$\hat{M}$

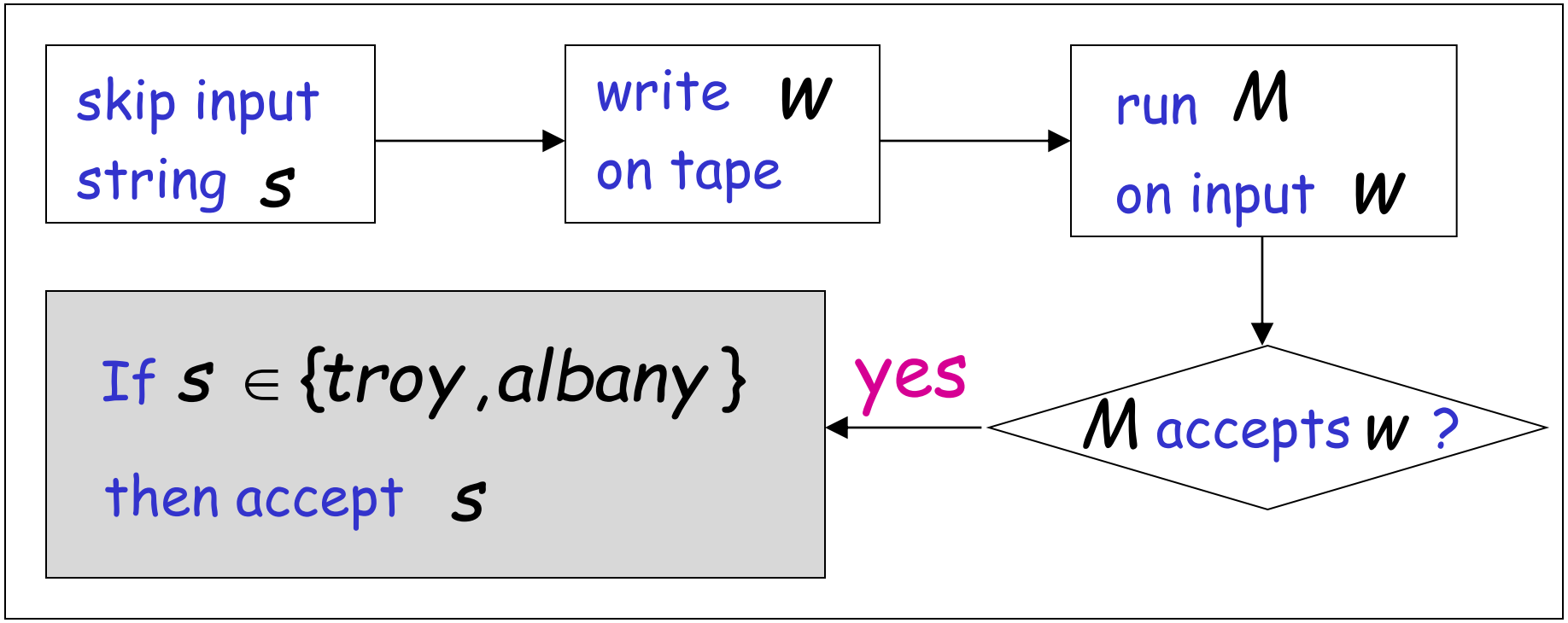


2 strings

M accepts w $\Rightarrow$ $L(\hat{M}) = \{troy, albany\}$

M does not accept w $\Rightarrow$ $L(\hat{M}) = \emptyset$ 0 strings

$\hat{M}$



Therefore:

M accepts w $\Longleftrightarrow$ $L(\hat{M})$ has size 2

Equivalently:

$\langle M, w \rangle \in AT_{TM}$ $\Longleftrightarrow$ $\langle \hat{M} \rangle \in SIZE2_{TM}$

END OF PROOF

RICE's Theorem

Undecidable problems:

- L is empty?
- L is regular?
- L has size 2?

This can be generalized to all non-trivial properties of Turing-acceptable languages

Non-trivial property:

A property P possessed by some Turing-acceptable languages but not all

Example: P_1 : L is empty?

YES $L = \emptyset$

NO $L = \{troy\}$

NO $L = \{troy, albany\}$

More examples of non-trivial properties:

P_2 : L is regular?

YES $L = \emptyset$

YES $L = \{a^n : n \geq 0\}$

NO $L = \{a^n b^n : n \geq 0\}$

P_3 : L has size 2?

NO $L = \emptyset$

NO $L = \{troy\}$

YES $L = \{troy, albany\}$

Trivial property:

A property P possessed by ALL Turing-acceptable languages

Examples:

P_4 : L has size at least 0?

True for all languages

P_5 : L is accepted by some Turing machine?

True for all Turing-acceptable languages

We can describe a property P as the set of languages that possess the property

If language L has property P then $L \in P$

Example: P : L is empty?

$P = \{\emptyset\}$

YES $L = \emptyset$

NO $L = \{troy\}$

NO $L = \{troy, albany\}$

Example: Suppose alphabet is $\Sigma = \{a\}$

P : L has size 1?

NO $\emptyset$

YES $\{\lambda\}$ $\{a\}$ $\{aa\}$ $\{aaa\}$ $\cdots$

NO $\{\lambda, a\}$ $\{\lambda, aa\}$ $\{a, aa\}$ $\cdots$

NO $\{\lambda, a, aa\}$ $\{aa, aaa, aaaa\}$ $\cdots$

$P = \{\{\lambda\}, \{a\}, \{aa\}, \{aaa\}, \{aaaa\}, \ldots\}$

Non-trivial property problem

Input: Turing Machine M

Question: Does $L(M)$ have the non-trivial property P? $L(M) \in P$?

Corresponding language:

$$PROPERTY_{TM} = \{\langle M \rangle : M \text{ is a Turing machine such that } L(M) \text{ has the non-trivial property } P, \text{ that is, } L(M) \in P\}$$

Rice's Theorem: $PROPERTY_{TM}$ is undecidable

(the non-trivial property problem is unsolvable)

Proof:

Reduce

A_{TM} (membership problem)

to

$PROPERTY_{TM}$ or $\overline{PROPERTY_{TM}}$

We examine two cases:

Case 1: $\emptyset \in P$

Examples: P : $L(M)$ is empty?

P : $L(M)$ is regular?

Case 2: $\emptyset \notin P$

Example: P : $L(M)$ has size 2?

Case 1: $\emptyset \in P$

Since P is non-trivial, there is a Turing-acceptable language X such that: $X \notin P$

Let M_X be the Turing machine that accepts X

Reduce

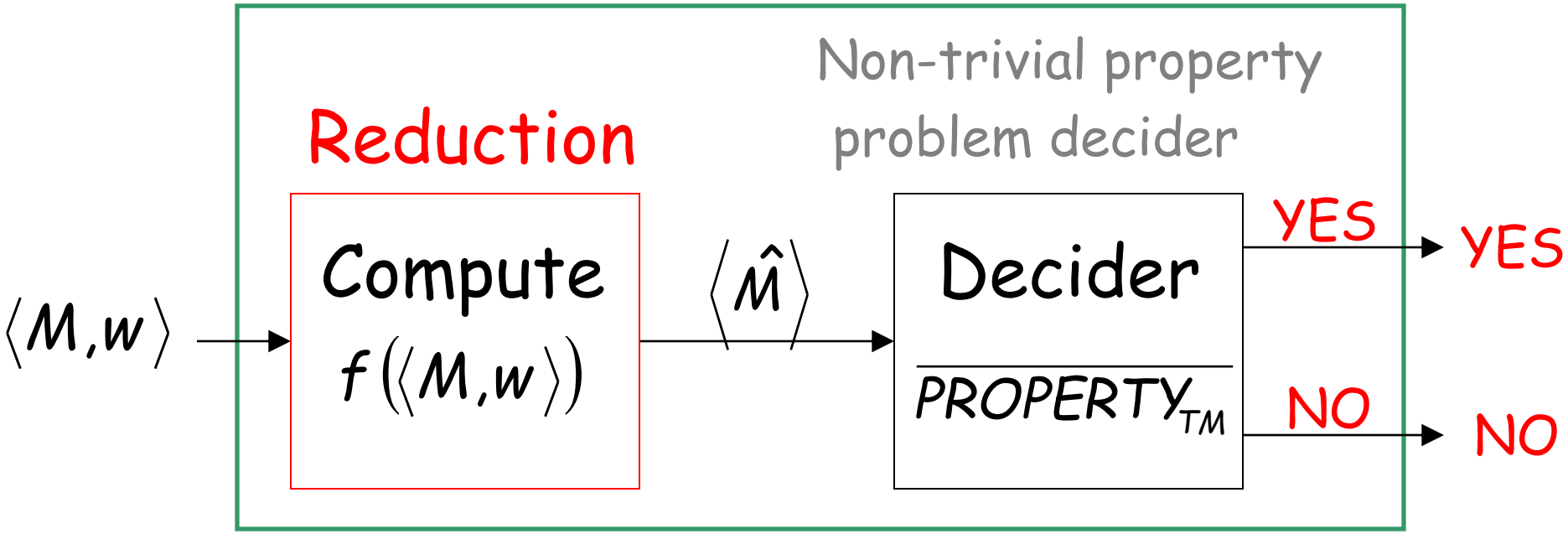
A_{TM} (membership problem)

to

$\overline{PROPERTY_{TM}}$

membership problem decider

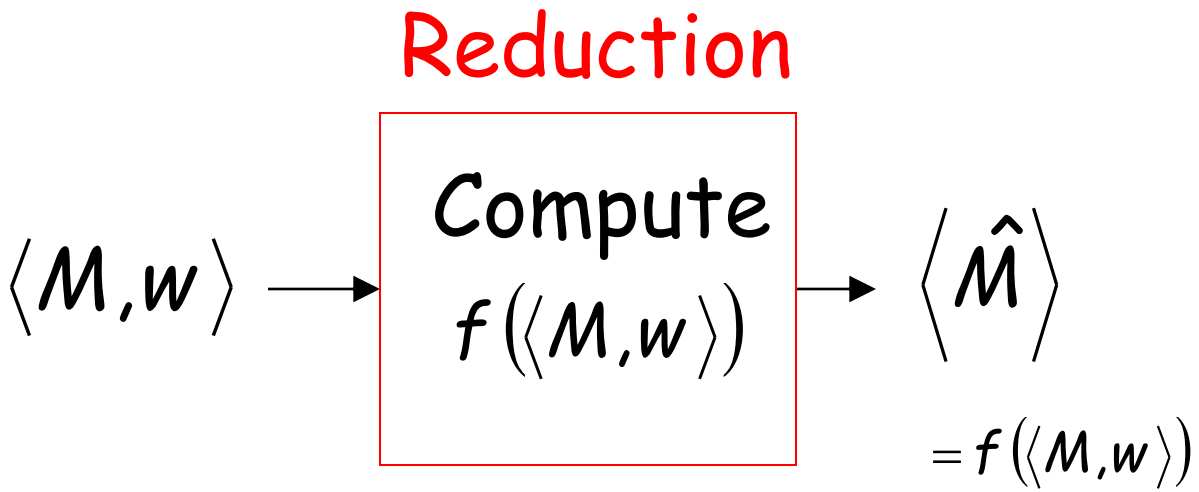
Decider for A_{TM}



Given the reduction,
if $\overline{PROPERTY_{TM}}$ is decidable,
then A_{TM} is decidable

A contradiction!
since A_{TM}
is undecidable

We only need to build the reduction:

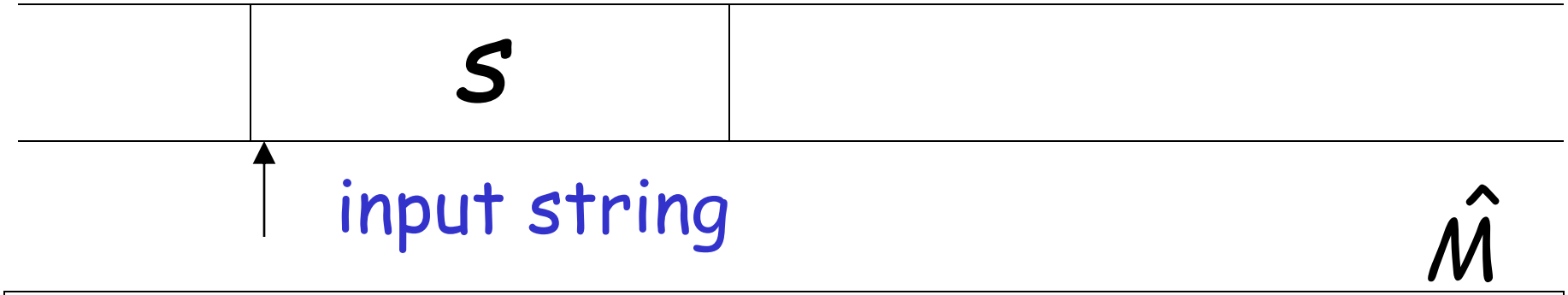


So that:

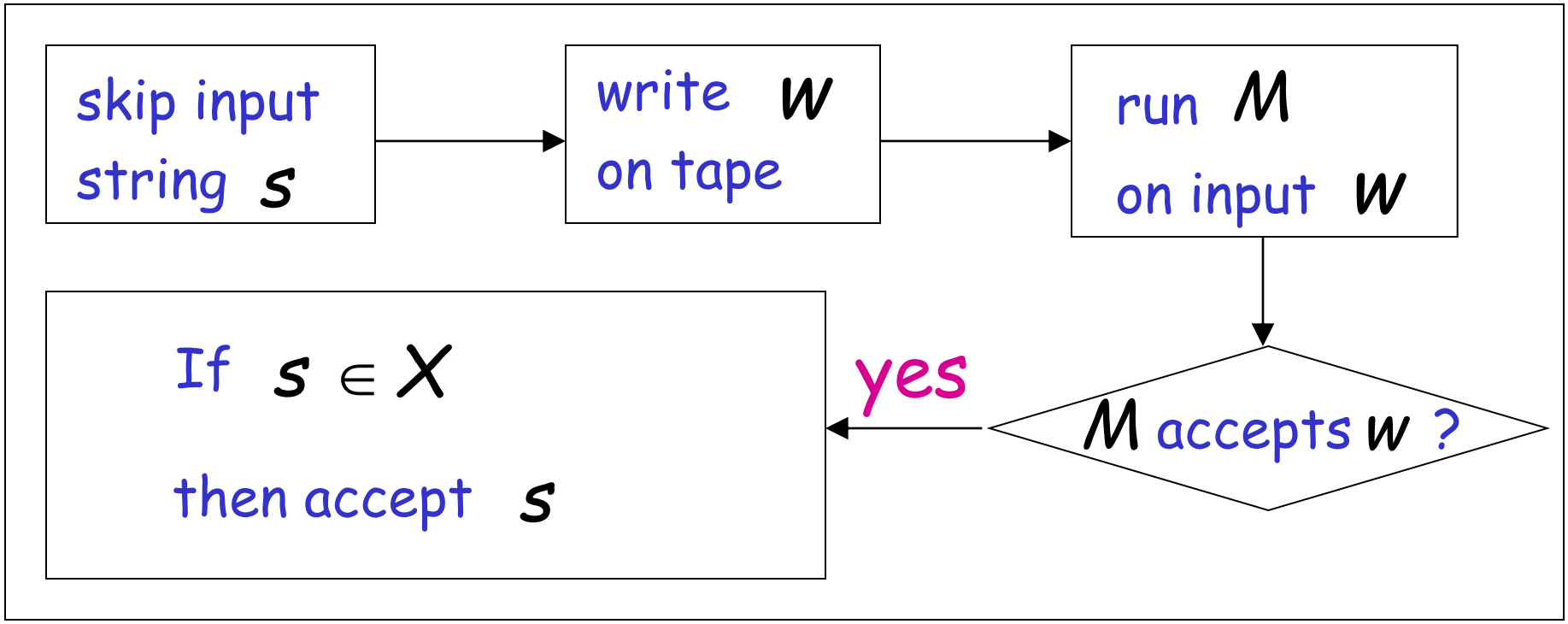
$$\langle M,w \rangle \in AT_{TM} \iff \langle \hat{M} \rangle \in \overline{PROPERTY_{TM}}$$

Construct $\langle \hat{M} \rangle$ from $\langle M, w \rangle$:

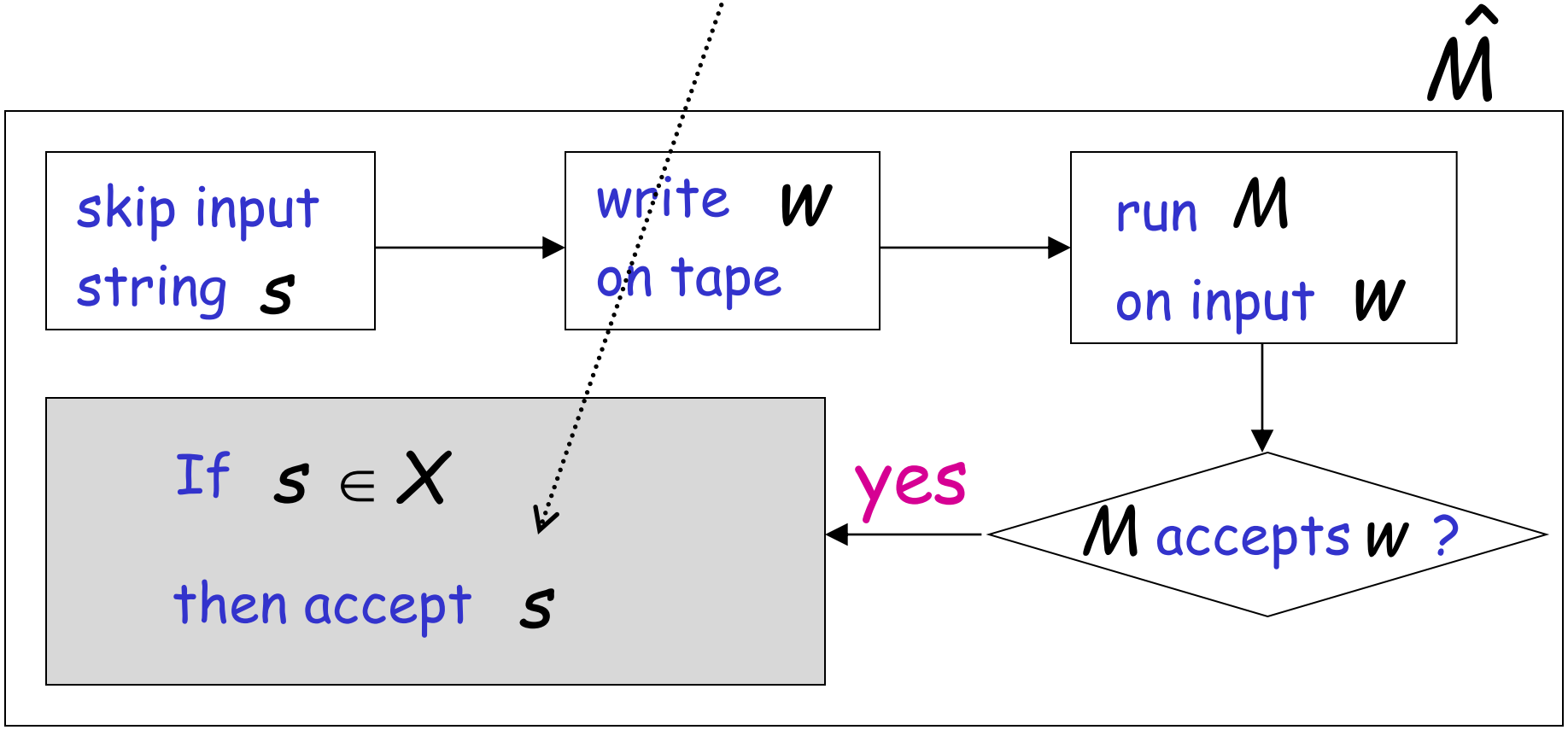
Tape of $\hat{M}$



$\hat{M}$

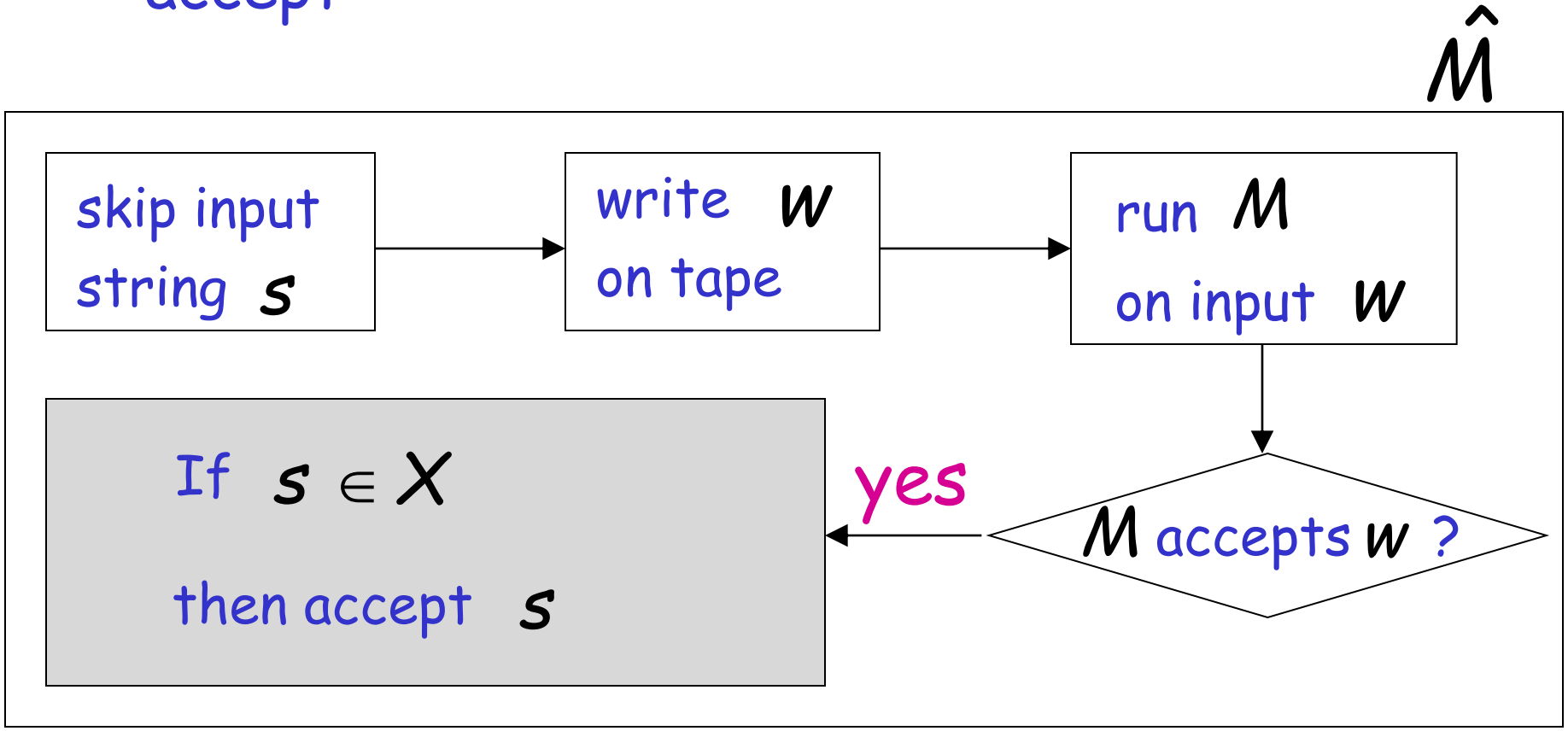


For this phase we can run machine M_X that accepts X, with input string s



M accepts w $\Rightarrow$ $L(\hat{M}) = X$ $\notin P$

M does not accept w $\Rightarrow$ $L(\hat{M}) = \emptyset$ $\in P$



Therefore:

M accepts w $\Longleftrightarrow$ $L(\hat{M}) \notin P$

Equivalently:

$\langle M, w \rangle \in AT_{TM}$ $\Longleftrightarrow$ $\langle \hat{M} \rangle \in \overline{PROPERTY_{TM}}$

Case 2: $\emptyset \notin P$

Since P is non-trivial, there is a Turing-acceptable language X such that: $X \in P$

Let M_X be the Turing machine that accepts X

Reduce

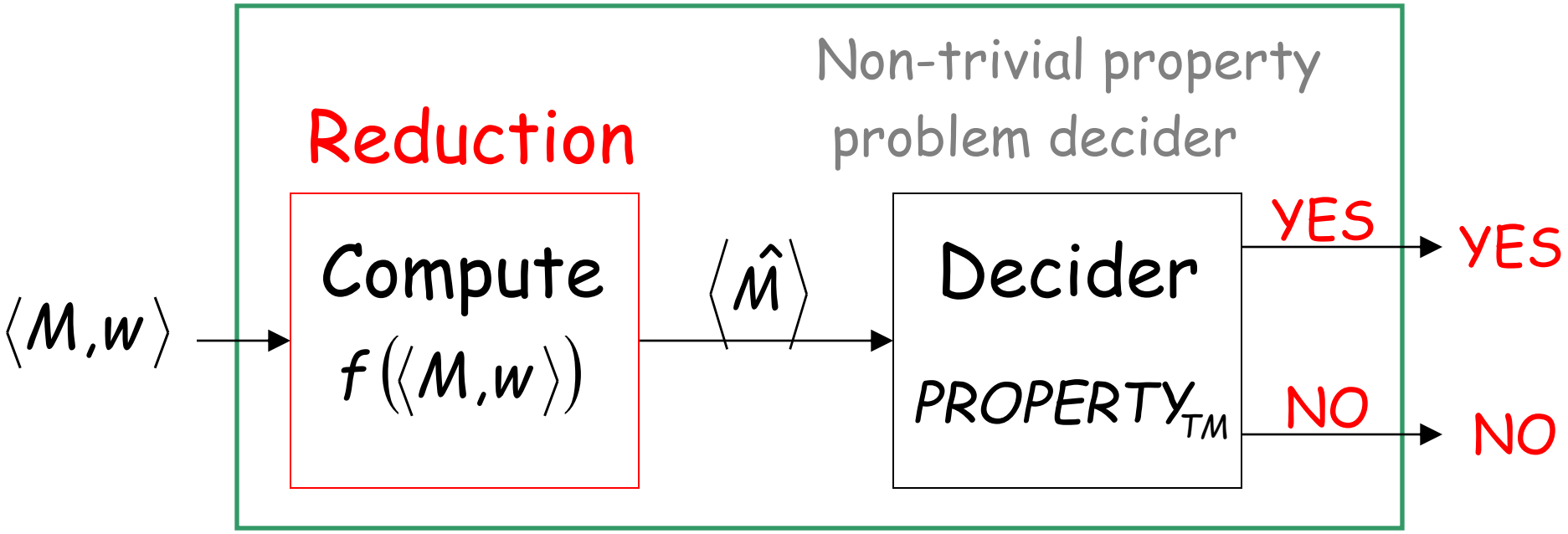
A_{TM} (membership problem)

to

$PROPERTY_{TM}$

membership problem decider

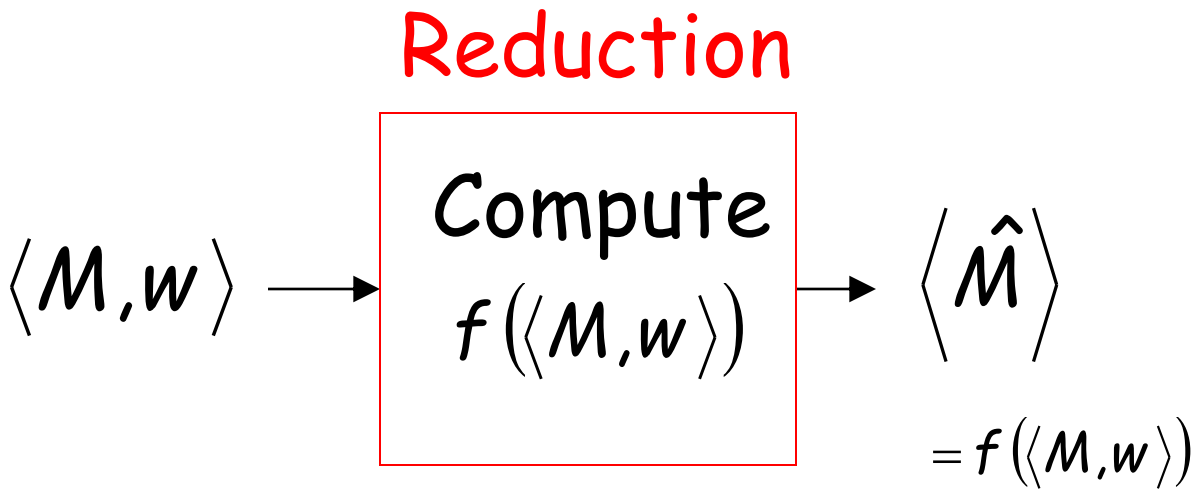
Decider for A_{TM}



Given the reduction,
if $PROPERTY_{TM}$ is decidable,
then A_{TM} is decidable

A contradiction!
since A_{TM}
is undecidable

We only need to build the reduction:

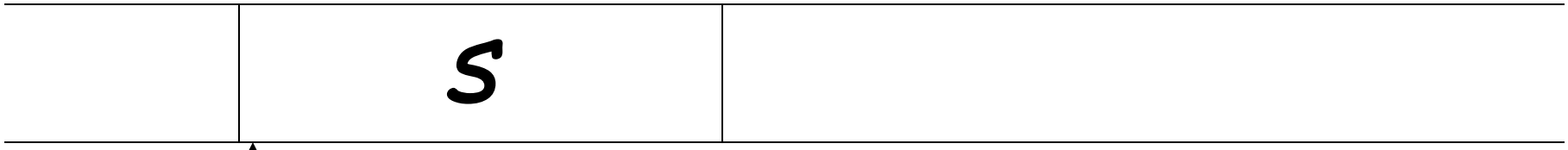


So that:

$$\langle M,w \rangle \in AT_{TM} \iff \langle \hat{M} \rangle \in PROPERTY_{TM}$$

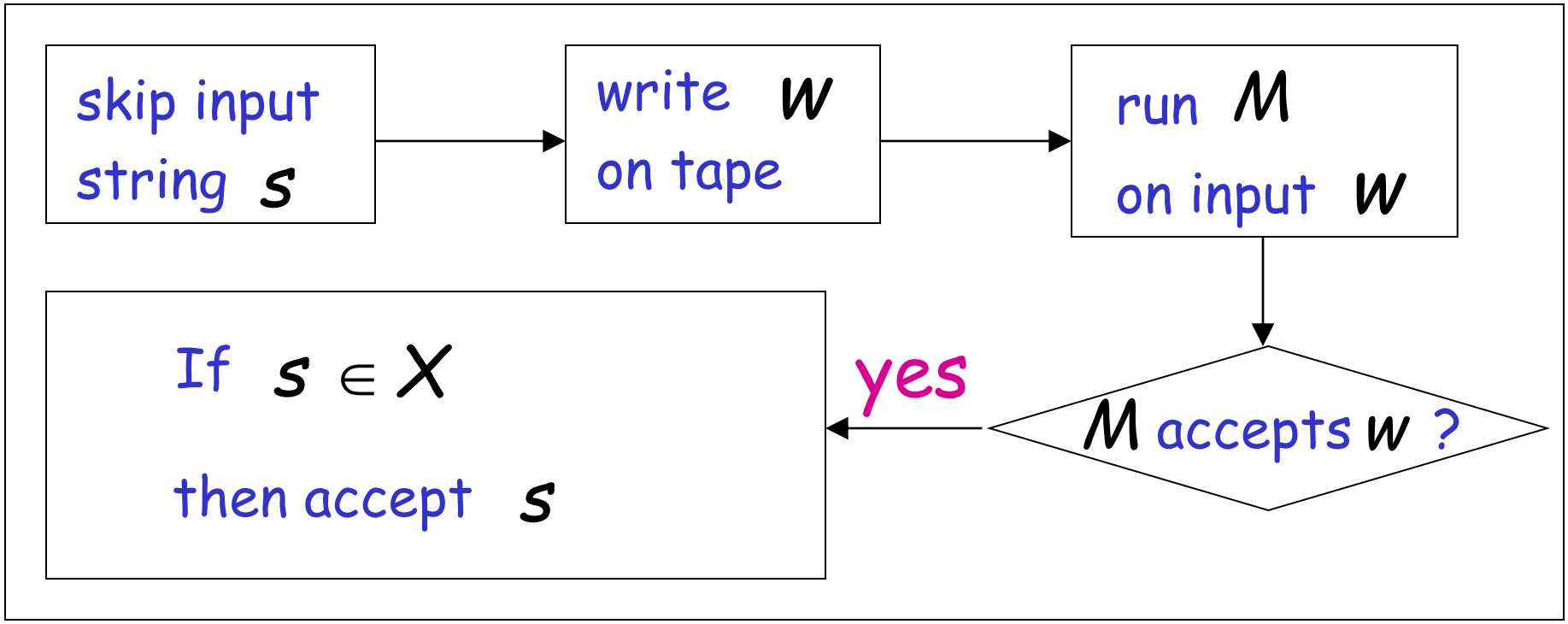
Construct $\langle \hat{M} \rangle$ from $\langle M, w \rangle$:

Tape of $\hat{M}$



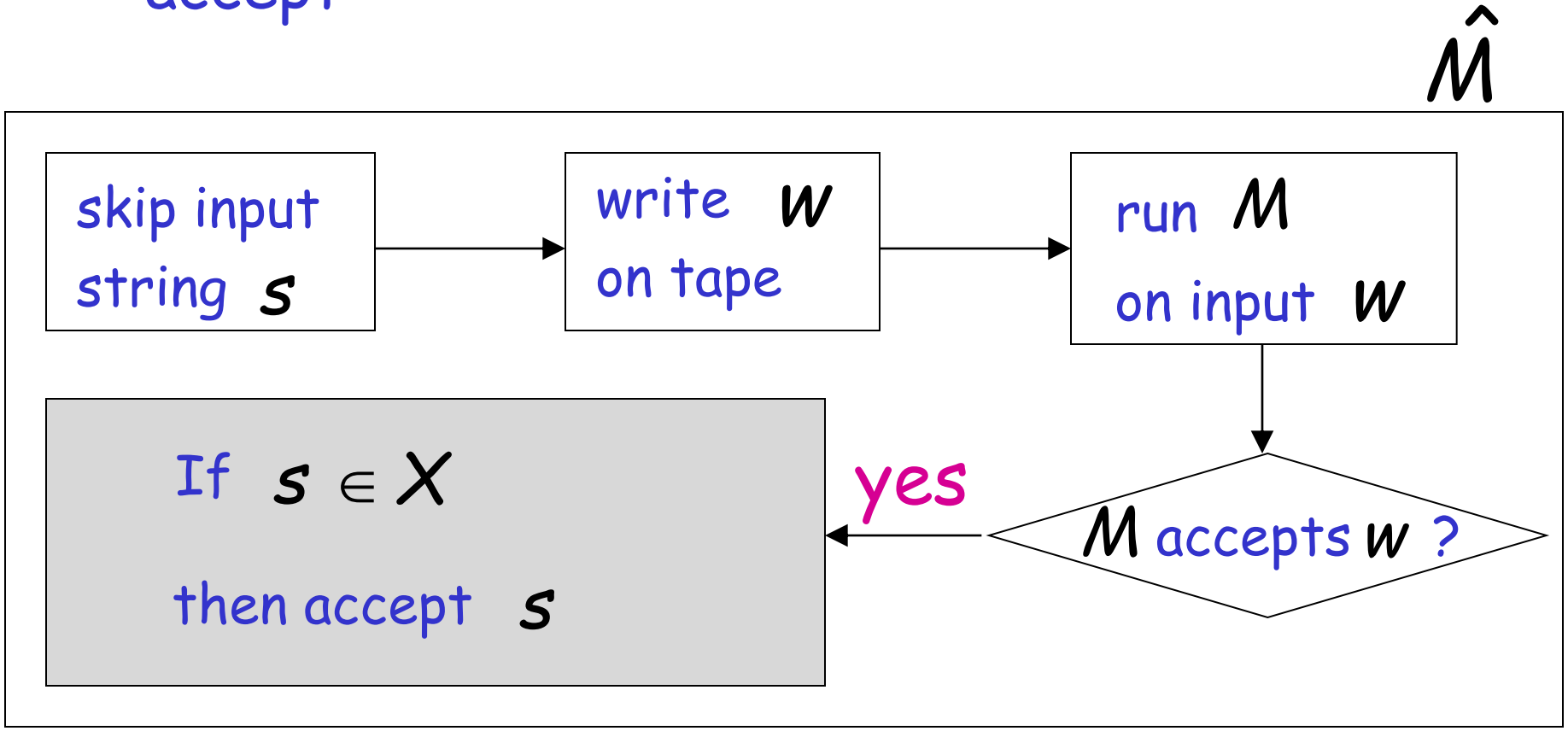
input string

$\hat{M}$



M accepts w $\Rightarrow$ $L(\hat{M}) = X$ $\in P$

M does not accept w $\Rightarrow$ $L(\hat{M}) = \emptyset$ $\notin P$



Therefore:

M accepts w $\Longleftrightarrow$ $L(\hat{M}) \in P$

Equivalently:

$\langle M, w \rangle \in AT_{TM}$ $\Longleftrightarrow$ $\langle \hat{M} \rangle \in PROPERTY_{TM}$

END OF PROOF